

# Faithful Tropical Explanations for Guiding Bi-Objective Quay-Crane and Speed-Adjustable-AGV Scheduling

Aayush Jha

August 14, 2026

## Abstract

This paper treats the bi-objective scheduling of quay cranes (QCs) and speed-adjustable automated guided vehicles (SA-AGVs) in a dual-cycling container terminal, where the two competing objectives are the makespan  $C_{\max}$  and the total AGV travel energy  $E$ . The reference method is the multi-population biased random-key genetic algorithm (mp-BRKGA) of Fontes and Homayouni. The question addressed is whether an explanation of why a schedule is good can be made faithful enough to describe the behaviour of the Pareto front and to steer the search. The model introduced here is a physics-fused heterogeneous graph attention surrogate (E-HGATv2) whose makespan is produced by an exact max-plus (tropical) longest-path layer. Because the makespan flows through the critical-path recurrence itself, the network’s gradient with respect to each transport leg is an exact binary critical-path indicator; this readout is denoted TAPE (Tropical Attribution for Physical Explanations). On 11 problem instances (synthetic dual-cycling instances and the published FSMJ loading benchmark, in both the uncoupled and the peak-power-coupled regime) TAPE recovers the true critical path with Jaccard 0.85–1.00 against the exact evaluator, whereas the learned attention weights are statistically uncorrelated with the true makespan levers ( $|\rho| \leq 0.18$ ) and, measured against the exact critical path from coarse hit-rate to fine ranking, are no better than random at any granularity. Routing the same signal back into an NSGA-II search makes the guided algorithm dominate mp-BRKGA, single-population BRKGA and an unguided control on hypervolume in 11/11 instances (Friedman  $\chi^2_4 = 35.1$ ,  $p = 4.5 \times 10^{-7}$ ; all post-hoc Holm-adjusted  $p \leq 0.004$ ), and the advantage widens with instance size on a  $N=10$ –160 ladder in both regimes, where at  $N=160$  the classical baselines reach at most 0.04 of the reference hypervolume within the same budget against 0.60 for the guided search. Aggregating TAPE over the approximated Pareto set with a trade-off-weighted score (Trade-off Criticality Score) yields a sparse description of how the binding bottleneck migrates along the front, and a single structural surrogate transfers this front composition to unseen instances (Pearson  $r = 0.945$  on a composition-spanning set). The same mathematical object thus explains the front, characterises its behaviour, and guides the optimiser.

## 1 Introduction

The container terminal scheduling problem treated here couples three decisions: assignment of container moves (tasks) to vehicles, sequencing of tasks on each vehicle and on each crane, and the speed level of every vehicle leg, which trades travel time against energy. Two objectives are minimised, the makespan  $C_{\max}$  and the total travel energy  $E$ . The reference method is the multi-population biased random-key genetic algorithm (mp-BRKGA) of Fontes and Homayouni [1].

**Table 1:** Kinematic and power constants per speed level (after [3]).

| Level   | $\alpha$ | empty $v$ (m/s) | empty $P$ (kW) | loaded $v$ (m/s) | loaded $P$ (kW) |
|---------|----------|-----------------|----------------|------------------|-----------------|
| Lower   | 0.8      | 4.8             | 7.8            | 2.4              | 11.7            |
| Nominal | 1.0      | 6.0             | 10.0           | 3.0              | 15.0            |
| Higher  | 1.2      | 7.2             | 13.2           | 3.6              | 19.8            |

Four requirements on such a solver are treated together here, and are referred to throughout by the labels below.

**R1. Feature engineering.** Derive the quantities that actually drive the objectives.

**R2. Landscape / feature-importance analysis.** On small instances, explain why particular solutions are Pareto-optimal in terms of the input variables (task order, vehicle speed, crane workload).

**R3. Feedback / guidance algorithm.** Use the knowledge gathered during the search to steer the optimiser (which neighbourhood to explore, which operator to apply).

**R4. Front behaviour.** Extract that knowledge in a way that captures the behaviour of the Pareto front itself.

A single mathematical object addresses all four: the gradient of an exact max-plus longest path embedded inside a learned surrogate. The signal is not an explainer fitted to a trained model after the fact; it is the dynamic program’s own subgradient, computed in the same backward pass that trains the network and consumed by the optimiser while it runs. It is the importance signal over the input variables (R1, R2), it is the steering signal fed back to NSGA-II (R3), and, aggregated over the front with a trade-off weight, it is a description of front behaviour (R4). The requirement throughout is faithfulness in a specific sense, that the signal identify the operations which set the makespan rather than produce a plausible-looking heat map. Learned attention weights read directly as explanations do not meet that requirement in this problem, and Section 5 quantifies the gap.

## 2 Problem formulation

### 2.1 Entities and decisions

An instance has a set of container moves (*tasks*)  $\mathcal{J} = \{0, \dots, N - 1\}$ . Each task  $j$  is bound to one quay crane  $qc(j)$  and one yard/storage station  $lu(j)$ , and is of one of two kinds: a *LOAD* (export: the loaded vehicle leg runs  $LU \rightarrow QC$ ) or an *UNLOAD* (import: the loaded leg runs  $QC \rightarrow LU$ ). Each crane processes its tasks one at a time with a handling time  $\tau_j$ . A fleet of  $|\mathcal{A}|$  SA-AGVs performs the transport. A decision specifies, for every task, (a) the assigned vehicle, (b) the position in that vehicle’s sequence and in its crane’s sequence, and (c) a discrete speed level for the *empty* leg (vehicle driving to the pickup) and for the *loaded* leg.

### 2.2 Physics of a leg

Three speed levels  $\alpha \in \{0.8, 1.0, 1.2\}$  scale a nominal speed; loaded and empty legs have distinct nominal speeds and power draws (Table 1, after [3, 1]). For a leg of length  $d$  at level  $\ell$ ,

$$t_{\text{leg}}(d, \ell) = \frac{d}{v_\ell}, \quad e_{\text{leg}}(d, \ell) = P_\ell \cdot t_{\text{leg}}(d, \ell), \quad (1)$$

i.e. energy is power  $\times$  time. A faster level shortens  $t$  but raises  $P$ , which is the source of the time–energy trade-off.

## 2.3 Objectives and the max-plus structure of the makespan

Total energy is simply additive over the chosen legs,

$$E = \sum_{j \in \mathcal{J}} (e_j^{\text{empty}} + e_j^{\text{loaded}}). \quad (2)$$

The makespan is more subtle. Fixing the assignment and sequencing turns the schedule into a *precedence graph*: task  $j$  cannot start its work until both its predecessor on the same vehicle and its predecessor on the same crane have finished. Following the corrected single-indexed timing of [2], the per-task completion obeys, for a LOAD,

$$c_j = \max(a_j^{\text{agv}} + t_j^{\text{empty}} + t_j^{\text{loaded}}, c_{\text{qcprev}(j)}) + \tau_j, \quad (3)$$

and for an UNLOAD,

$$c_j = \max(a_j^{\text{agv}} + t_j^{\text{empty}}, c_{\text{qcprev}(j)} + \tau_j) + t_j^{\text{loaded}}, \quad (4)$$

with  $a_j^{\text{agv}}$  the time the vehicle becomes free for  $j$ . Because every completion is a *maximum of sums* along the vehicle and crane chains, the makespan

$$C_{\max} = \max_j c_j \quad (5)$$

is exactly a *longest path* in the precedence graph under the algebra  $(\max, +)$ —the *tropical* or *max-plus* semiring [11]. This is the single structural fact the whole method exploits: the longest path is the *critical path*, and the objective is determined by the activities lying on it.

Section 3.5 embeds this recurrence as a differentiable layer over *predicted* durations, which is what makes the path available for schedules the exact evaluator has not been run on, and makes it a gradient that trains the network that produced them.

## 2.4 The peak-power-coupled regime

The uncoupled problem above has a closed-form (longest-path) makespan. The second regime adds an instantaneous peak-power constraint, following the speed-adjustable peak-power scheduling model of Homayouni and Fontes [4]: the fleet shares a single *instantaneous peak-power budget*  $P_{\max}$ , and the total power of all AGV *travel legs* running at any instant may not exceed  $P_{\max}$  (quay-crane handling draws none). Their model is stated for job-shop machines; the transfer to AGV travel legs in the terminal setting is made here. Any  $P_{\max} \geq \max_{\ell} \varepsilon_{\ell}$  ( $= 19.8 \text{ kW}$  from Table 1; we use  $30 \text{ kW}$ ) keeps every leg individually feasible.

**Activity model.** Each task  $j$  expands into power-drawing legs and zero-power timing nodes whose precedence *reproduces the uncoupled recurrence of Section 2.3 exactly*. The empty (E) and loaded (L) legs draw power  $e_{\ell}, \varepsilon_{\ell}$  for their durations  $t_j^e, t_j^l$ ; the zero-duration nodes are the AGV release R (at the quay pickup, for a LOAD) and the completion C (for an UNLOAD), and node H carries the handling time  $\tau_j$ . Within a task the order is  $E \rightarrow L \rightarrow R \rightarrow H$  (LOAD) or  $E \rightarrow C \rightarrow L$  (UNLOAD), with R/C gated by the crane chain so that, *absent the cap*, the realised completions equal the closed-form  $c_j$  of Eqs. (LOAD/UNLOAD); AGV- and crane-chain arcs link consecutive tasks.

**Resolution.** A power leg may *start* only once its precedence is met and starting it keeps the sum of concurrently-running leg powers  $\leq P_{\max}$ ; otherwise it waits for a running leg to finish and free budget. The schedule is realised by a deterministic *parallel schedule-generation scheme*: at each completion event, ready legs are started greedily in priority order—earliest available, then task index, then empty-before-loaded—until the budget is exhausted. This makes  $(C_{\max}, E)$  a

*well-defined, exact* function of the decision  $z$  (energy is unchanged, being timing-independent), but the makespan is *no longer* a max-plus longest path: the cap injects disjunctive *power-wait* arcs (a blocker leg  $\rightarrow$  the leg it delays) that no closed form captures. When  $P_{\max}$  is large enough that no leg ever waits, the scheme provably reduces to the uncoupled evaluator (a unit test asserts this), so the coupled problem is a clean *superset* of the validated uncoupled one.

**Scope of the regime.** Two properties delimit it. (i) The power arbitration is a *fixed dispatch policy* (the priority rule above) rather than a decision variable; the regime optimises the assignment, order and speeds under that policy. (ii) The peak-power model is published for job-shop scheduling [4] and the uncoupled terminal recurrence is published in [2], but their combination—a peak-power budget over AGV travel legs in the dual-cycling terminal—has no published instance set, so the coupled results are reported on the SD- $N$ -C instances of Appendix B. This is the regime in which the makespan admits no closed form and the binding leg is not evident by inspection, and therefore the regime in which a differentiable surrogate with exact gradients (TAPE) is required; accordingly, the fused head carries an explicit power-wait term (Section 3.5).

### 3 Method

**Overview.** There is no labelled training corpus. An exact simulator—the *evaluator* (Section 3.6)—scores any candidate schedule and, as a by-product, emits the duration of every transport leg; a graph network then learns to imitate those durations. Concretely (Figure 1): a search proposal is decoded into a feasible schedule (Section 3.2), recast as a heterogeneous graph (Section 3.3) and embedded by the E-HGATv2 surrogate (Section 3.4); projection heads predict each leg’s duration, which an *exact* max-plus dynamic program (Section 3.5) recomposes into the makespan. Because the makespan flows through the genuine critical-path recurrence, differentiating it returns an *exact* indicator of which operations bind the schedule—the TAPE explanation (Section 3.7). That same signal biases the search’s variation operators and screens its offspring (Section 3.8) and, aggregated over the Pareto front, describes how the binding bottleneck migrates (Section 3.9). The rest of this section dissects each block with the notation of Table 2.

#### 3.1 Decision variables and the search space

An instance has  $N$  transport tasks  $\mathcal{J} = \{1, \dots, N\}$ , a fleet of  $A$  speed-adjustable AGVs,  $K$  quay cranes, and  $V$  admissible speed levels per leg. A *solution* is the tuple

$$z = (\alpha, \sigma, \omega^e, \omega^l), \quad \alpha : \mathcal{J} \rightarrow \{1, \dots, A\}, \quad \sigma \in \mathfrak{S}_N, \quad \omega^e, \omega^l \in \{1, \dots, V\}^N, \quad (6)$$

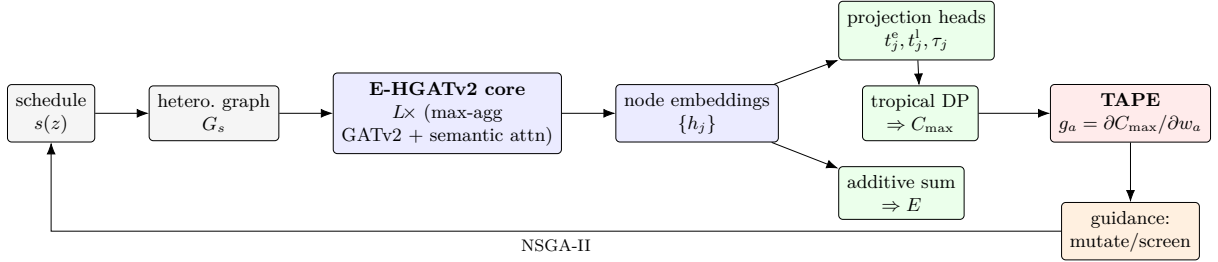
i.e. an assignment  $\alpha$  of each task to a vehicle, a processing order  $\sigma$ , and a speed level for each empty (e) and loaded (l) leg. The decoder of Section 3.2 turns  $z$  into a feasible schedule  $s(z)$  whose physical evaluation returns the objective pair  $f(s) = (C_{\max}(s), E(s))$ . The cardinality of the feasible set,

$$|\mathcal{Z}| = \underbrace{A^N}_{\text{assignment}} \cdot \underbrace{N!}_{\text{order}} \cdot \underbrace{V^{2N}}_{\text{speeds}}, \quad (7)$$

grows super-exponentially (already  $> 10^{40}$  at  $N=20, A=3, V=3$ ), and the underlying assignment–sequencing problem is NP-hard [1]; exact MILP is intractable beyond a few dozen tasks. We therefore search with the population-based metaheuristic NSGA-II [5] and concentrate our contribution on where in  $\mathcal{Z}$  the search spends its budget: a faithful, physics-anchored explanation of *which decisions bind the objectives* is used to bias the variation operators and to screen offspring (Sections 3.8).

**Table 2:** Notation used throughout the Method.

| Symbol                                                  | Meaning                                                                 |
|---------------------------------------------------------|-------------------------------------------------------------------------|
| $\mathcal{J} = \{1, \dots, N\}$                         | transport tasks                                                         |
| $A, K, V$                                               | # AGVs, # quay cranes, # speed levels per leg                           |
| $z = (\alpha, \sigma, \omega^e, \omega^l)$              | a solution (Eq. 6)                                                      |
| $s(z), f(s) = (C_{\max}, E)$                            | decoded schedule and its objective pair                                 |
| $G_s = (\mathcal{V}, E^{\text{agv}}, E^{\text{qc}})$    | heterogeneous schedule graph                                            |
| $x_j \in \mathbb{R}^{d_0}, u_{ij} \in \mathbb{R}^{d_e}$ | task-node and arc input features                                        |
| $h_j^{(\ell)} \in \mathbb{R}^d$                         | embedding of task $j$ after layer $\ell$                                |
| $\mathcal{N}_r(j)$                                      | resource- $r$ predecessors of $j$ ( $r \in \{\text{agv}, \text{qc}\}$ ) |
| $\beta_j^r$                                             | semantic (cross-resource) attention of $j$ on $r$                       |
| $t_j^e, t_j^l$                                          | empty/loaded leg times of task $j$                                      |
| $e_j^e, e_j^l$                                          | empty/loaded leg energies of task $j$ ( $E = \sum_j e_j^e + e_j^l$ )    |
| $\tau_j$                                                | crane handling delay of task $j$                                        |
| $M$                                                     | # sampled schedules used to train the surrogate                         |
| $G_{\text{dag}} = (\mathcal{N}, \mathcal{A}, b, w)$     | event DAG: node weights $b$ , arc weights $w$                           |
| $\pi^*$                                                 | critical (longest) path in $G_{\text{dag}}$                             |
| $g_a = \partial C_{\max} / \partial w_a$                | TAPE attribution of arc $a$                                             |



**Figure 1:** Pipeline. The E-HGATv2 core embeds a decoded schedule; projection heads predict local physical attributes that an *exact* max-plus dynamic program composes into  $C_{\max}$  (energy is an exact additive sum). Differentiating the DP yields the TAPE arc attributions  $g_a$ , which both *explain* the schedule and *steer* the next NSGA-II generation.

### 3.2 Random-key representation and the decoder

We use the biased random-key encoding [1]: a chromosome is a vector  $\chi \in [0, 1]^{4N}$  split into four length- $N$  blocks. The deterministic decoder  $\mathcal{D} : [0, 1]^{4N} \rightarrow \mathcal{Z}$  reads task order from the argsort of the first block, vehicle assignment from  $\lceil A \chi^{(2)} \rceil$ , and the two speed levels from  $\lceil V \chi^{(3)} \rceil, \lceil V \chi^{(4)} \rceil$ ; structural feasibility (one vehicle per task, crane order from the fixed bay layout) is enforced during decoding. The optimiser is elitist NSGA-II [5]; our contribution lies in *how its mutation and screening are guided*.

### 3.3 Heterogeneous graph representation of a schedule

A decoded schedule  $s(z)$  is encoded as a directed heterogeneous graph  $G_s = (\mathcal{V}, E^{\text{agv}}, E^{\text{qc}})$  in which every transport task is a node  $j \in \mathcal{V}$  and the two resource chains are the two arc types:

- a **task-node feature**  $x_j \in \mathbb{R}^{d_0}$ ,  $d_0=4$ , holds the handling time, task kind and origin/destination descriptors;
- an **AGV arc**  $(i \rightarrow j) \in E^{\text{agv}}$  exists iff task  $i$  is the immediate vehicle-predecessor of  $j$  under  $\alpha, \sigma$ ; its feature  $u_{ij} \in \mathbb{R}^{d_e}$  ( $d_e=3$ ) carries the leg travel time and the empty/loaded leg energies;
- a **QC arc**  $(i \rightarrow j) \in E^{\text{qc}}$  exists iff  $i$  immediately precedes  $j$  on the same crane.

This graph is the precedence DAG whose longest path is the makespan (Section 2.3); the representation is therefore *isomorphic to the objective’s combinatorial structure*. A key fact exploited below is that in any *resolved* schedule each task has at most one AGV predecessor and at most one QC predecessor, i.e.  $|\mathcal{N}_{\text{agv}}(j)| \leq 1$  and  $|\mathcal{N}_{\text{qc}}(j)| \leq 1$ .

### 3.4 E-HGATv2: a (max, +)-aligned message-passing surrogate

A graph neural network is a parametric map on graphs that, in  $L$  rounds, lets each node recompute its state  $h_j^{(\ell)} \in \mathbb{R}^d$  from its neighbours’ states. We design the update to mirror the makespan recurrence (18), whose two operators are a *within-resource* max and a *cross-resource* max.

**Elementary operators.** We use four standard building blocks. For a vector  $z \in \mathbb{R}^m$ , the rectifier  $\text{ReLU}(z) = \max(0, z)$  acts coordinatewise and the leaky variant  $\text{LeakyReLU}(z) = \max(\eta z, z)$  ( $\eta=0.2$ ) keeps a small negative slope; the softmax  $(z)_k = e^{z_k} / \sum_{k'} e^{z_{k'}}$  maps scores to a probability vector. A *multilayer perceptron* composes affine maps with ReLU,  $\text{MLP}(z) = W_2 \text{ReLU}(W_1 z + b_1) + b_2$ . A *read-out* pools a variable-size set of node states  $\{h_j\}_{j \in V}$  into one vector with the permutation-invariant operators mean-pool  $(\{h_j\}) = \frac{1}{|V|} \sum_j h_j$ , max-pool  $(\{h_j\}) = \max_j h_j$  (coordinatewise), and sum-pool  $(\{u_a\}) = \sum_a u_a$ . The two max operators are what align the network with the (max, +) objective.

The encoder is  $h_j^{(0)} = \text{ReLU}(W_0 x_j)$ , and each of the  $L$  layers ( $L=3$ ,  $d=64$ , 4 heads) applies, per task  $j$ :

**(i) Within-resource attention and max reduction.** For each relation  $r \in \{\text{agv}, \text{qc}\}$  we run a GATv2 operator [7]. GATv2 scores an arc  $(i \rightarrow j)$  after the nonlinearity, which (unlike GAT) makes the attention a universal function of the endpoint–edge triple:

$$e_{ij}^r = \mathbf{a}_r^\top \text{LeakyReLU}(W_r [h_i^{(\ell-1)} \parallel h_j^{(\ell-1)} \parallel u_{ij}]), \quad \alpha_{ij}^r = \text{softmax}_{i \in \mathcal{N}_r(j)} e_{ij}^r. \quad (8)$$

The relation message is taken with an *elementwise maximum* over predecessors,

$$m_j^r = \max_{i \in \mathcal{N}_r(j)} \alpha_{ij}^r \Theta_r [h_i^{(\ell-1)} \parallel u_{ij}], \quad (9)$$

where  $W_r \in \mathbb{R}^{2d+d_e}$  and  $\mathbf{a}_r$  are the GATv2 score parameters and  $\Theta_r: \mathbb{R}^{d+d_e} \rightarrow \mathbb{R}^d$  is the per-relation value transform (realised as the 4 attention heads of width  $d/4$ , concatenated), so  $m_j^r \in \mathbb{R}^d$ . Equation (9) is the (max, +) reduction: a node inherits the *binding* predecessor, exactly as the longest path does, rather than the additive blend  $\sum_i \alpha_{ij}(\cdot)$  of a standard attention layer. (Because  $|\mathcal{N}_r(j)| \leq 1$  in a resolved schedule, the per-relation softmax is degenerate— $\alpha_{ij}^r \equiv 1$ —so the *non-degenerate* selection happens across relations, next.)

**Choice of GATv2 over GAT.** The original GAT [8] scores an arc as  $\text{LeakyReLU}(\mathbf{a}^\top [W h_i \parallel W h_j])$ , applying the learned vector  $\mathbf{a}$  and the matrix  $W$  before the nonlinearity. Brody et al. [7] prove this yields *static attention*: the induced ranking of source nodes is global—the most-attended key is the *same for every query node*  $j$ . That is the wrong inductive bias here, because which predecessor (and which resource chain) *gates* a task is intrinsically *query-dependent*: a long empty leg binds one task yet is slack for another, depending on the rest of its chain. GATv2 moves the nonlinearity *inside* the score,  $e_{ij}^r = \mathbf{a}_r^\top \text{LeakyReLU}(W_r [h_i \parallel h_j \parallel u_{ij}])$ , giving *dynamic attention* that can realise a different ranking per task. The cross-resource gating (10) that mirrors  $\max(\text{agv-ready}, \text{qc-ready})$  is a per-task selection of the binding chain, which the dynamic form supports and the static one does not: a static-attention GAT cannot represent “task  $j$  is AGV-bound while task  $k$  is QC-bound.”

(ii) **Cross-resource semantic attention.** A task completes when the *later* of its two resource chains is ready,  $\max(\text{agv-ready}, \text{qc-ready})$ . We make this argmax differentiable with a semantic (HAN-style) attention [9] over the two relation messages. With a shared scorer  $(W_s, \mathbf{q})$ ,

$$s_j^r = \mathbf{q}^\top \tanh(W_s m_j^r), \quad (\beta_j^{\text{agv}}, \beta_j^{\text{qc}}) = \text{softmax}(s_j^{\text{agv}}, s_j^{\text{qc}}), \quad (10)$$

$$m_j = \beta_j^{\text{agv}} m_j^{\text{agv}} + \beta_j^{\text{qc}} m_j^{\text{qc}}, \quad h_j^{(\ell)} = h_j^{(\ell-1)} + \text{ReLU}(m_j), \quad (11)$$

where  $s_j^{\text{qc}} \rightarrow -\infty$  if  $j$  has no crane predecessor, and the residual connection lets criticality propagate across the  $L$  hops of the DAG. The scalar  $\beta_j^r$  is exactly “how much resource  $r$  gates task  $j$ ”—and is the per-arc weight one is tempted to read as criticality.

(iii) **Physics-matched read-out.** Inputs are standardised once at the encoder by frozen statistics  $\tilde{x}_j = (x_j - \mu_{\text{node}}) \oslash \sigma_{\text{node}}$  and  $\tilde{u}_a = (u_a - \mu_{\text{agv}}) \oslash \sigma_{\text{agv}}$  ( $\oslash$  is elementwise division). The two objectives have different algebra, so the read-out forms three pooled vectors from the final node states  $h_j^{(L)} \in \mathbb{R}^d$ —two for the longest-path makespan and one for the additive energy:

$$g_{\text{mean}} = \frac{1}{N} \sum_j h_j^{(L)} \in \mathbb{R}^d, \quad g_{\text{max}} = \max_j h_j^{(L)} \in \mathbb{R}^d \text{ (coordinatewise)}, \quad g_E = \sum_{a \in E^{\text{agv}}} \tilde{u}_a \in \mathbb{R}^{d_e}. \quad (12)$$

The energy uses a *sum* pool because a max pool would discard the additive arc contributions. Concatenating,  $z = [g_{\text{mean}} \parallel g_{\text{max}} \parallel g_E] \in \mathbb{R}^{2d+d_e}$ , a two-layer head regresses the standardised objective vector and a buffer de-normalises it to physical units:

$$\hat{y} = W_2 \text{ReLU}(W_1 z + b_1) + b_2 \in \mathbb{R}^2, \quad (\widehat{C_{\text{max}}}, \widehat{E}) = \hat{y} \odot \sigma_y + \mu_y, \quad (13)$$

with  $W_1 \in \mathbb{R}^{d \times (2d+d_e)}$ ,  $W_2 \in \mathbb{R}^{2 \times d}$  and  $\odot$  the Hadamard product. Training minimises the standardised MSE  $\sum_s \|\hat{y}(s) - (y(s) - \mu_y) \oslash \sigma_y\|_2^2$  over decoded schedules  $s$  (so the makespan and energy losses are balanced despite their different scales). This plain regressor attains  $R^2 \approx 0.99$  on held-out schedules in the uncoupled regime (Section 5).

**Proposition 1** (Inductive-bias alignment). *Replacing the additive aggregation of a standard attention/MPNN layer with the  $(\max, +)$  pair (9)–(10) makes the layer’s two reductions coincide with the two max operators of the makespan recurrence (18). A sum-aggregation network must instead approximate max with depth/width, whereas E-HGATv2 represents it exactly in the limit  $\beta \rightarrow \{0, 1\}$ .*

**Limits of the attention weights as an explanation.** A natural reading of  $\beta_j^r$  from (10) takes it as a measure of how critical arc  $r$  of task  $j$  is. Section 5 shows this reading to be unfaithful: the  $\beta$  ranking is statistically uncorrelated with the true critical path (the softmax is a smooth surrogate, not the argmax, and it is trained only to fit the *scalar*  $C_{\text{max}}$ , with no constraint tying it to the binding path). This is the gap the fused head closes.

### 3.5 Physics-fused tropical head

The read-out (13) regresses the scalar  $C_{\text{max}}$  through a generic MLP. The Jacobian of that MLP *smears* sensitivity across every pooled feature, so an input-attribution of  $C_{\text{max}}$  has no reason to coincide with the binding path. The *fused* model keeps the trained E-HGATv2 core as a frozen feature extractor but replaces the makespan MLP with the genuine physics in three steps: (i) project embeddings to local physical attributes, (ii) assemble the event DAG, (iii) compose  $C_{\text{max}}$  with an exact differentiable max-plus dynamic program.

**(i) Projection heads.** Let task  $j$  have its unique incoming AGV arc ( $i \rightarrow j$ ) with standardised arc prior  $\tilde{u}_{ij}$ , and let  $h_i, h_j$  be the frozen-core embeddings. The heads emit local attributes in de-normalised physical units (buffers  $\mu_\bullet, \sigma_\bullet$  from training statistics,  $[\cdot]_+ = \text{ReLU}$ ):

$$(t_j^e, t_j^l) = [\text{MLP}_{\text{leg}}([h_i \parallel h_j \parallel \tilde{u}_{ij}]) \odot \sigma_{\text{leg}} + \mu_{\text{leg}}]_+, \quad (14)$$

$$\tau_j = [\text{MLP}_{\text{delay}}([h_j \parallel \tilde{x}_j^{\text{hand}}]) \sigma_\tau + \mu_\tau]_+. \quad (15)$$

Here  $\text{MLP}_{\text{leg}}: \mathbb{R}^{2d+d_e} \rightarrow \mathbb{R}^2$  and  $\text{MLP}_{\text{delay}}: \mathbb{R}^{d+1} \rightarrow \mathbb{R}$  (the extra input being the standardised handling prior  $\tilde{x}_j^{\text{hand}}$ ). The outer  $[\cdot]_+ = \text{ReLU}$  enforces the physical non-negativity  $t_j^e, t_j^l, \tau_j \in \mathbb{R}_{\geq 0}$  that the tropical layer (18) requires: a negative leg time or handling delay would shorten  $C_{\text{max}}$  below the binding path and so corrupt the longest-path semantics. The heads thus map abstract embeddings to a strictly non-negative physical parametrisation  $(b_v, w_a)$  of the event DAG. The leg *energies*  $e_j^e, e_j^l$  are read exactly from the arc features (not learned), so energy is a strictly linear head,

$$E = \sum_{j \in \mathcal{J}} (e_j^e + e_j^l), \quad \frac{\partial E}{\partial e_j^\bullet} = 1. \quad (16)$$

In the coupled regime an additional head predicts a per-leg power wait from a contention vector  $c_j \in \mathbb{R}^{d_c}$  read off the previous timing iterate,

$$\delta_j^\bullet = \text{MLP}_{\text{wait}}([h_i \parallel h_j \parallel \tilde{u}_{ij} \parallel c_j]), \quad \text{MLP}_{\text{wait}}: \mathbb{R}^{2d+d_e+d_c} \rightarrow \mathbb{R}^2, \quad (17)$$

so the effective leg weight becomes  $t_j^\bullet + \delta_j^\bullet$  and the prediction is the fixed point of an unrolled timing  $\rightarrow$  contention  $\rightarrow$  wait recursion (the differentiable analogue of the simulator’s iterative power resolution).

**(ii) Event DAG.** The attributes instantiate a weighted DAG  $G_{\text{dag}} = (\mathcal{N}, \mathcal{A}, b, w)$  whose nodes are task events, whose node weights  $b_v$  are handling delays  $\tau$ , and whose arc weights  $w_a$  are the (effective) leg times—exactly the precedence structure of Section 2.3.

**(iii) Tropical longest-path layer.** The completion time of every node solves, in topological order,

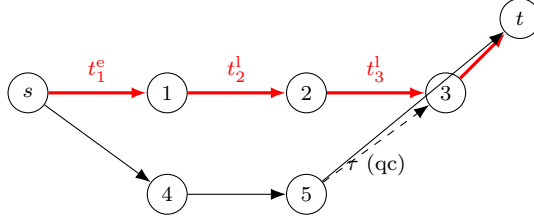
$$x_v = b_v + \max_{u: (u \rightarrow v) \in \mathcal{A}} (x_u + w_{uv}), \quad C_{\text{max}} = \max_{v \in \mathcal{N}_{\text{end}}} x_v, \quad (18)$$

and the layer stores, per node, the maximising in-arc  $a^*(v) = \arg \max_{u \rightarrow v} (x_u + w_{uv})$ .

**Proposition 2** (Exactness of the makespan head). *With  $b, w$  set to the true handling delays and leg times, (18) equals the physical makespan of Section 2.3; the recurrence is the  $(\max, +)$  rewriting of the LOAD/UNLOAD completions  $c_j$ . Hence the only approximation in the fused makespan is the heads’ prediction of the local attributes—the composition is exact.*

**Scope of the term “exact”.** Proposition 2 concerns the *composition*, not the physics: given the heads’ predicted leg attributes, TAPE returns the critical path of the surrogate’s max-plus DP exactly, being that DP’s own subgradient. The attribution’s fidelity to the *physical* critical path is therefore inherited from the heads’ accuracy. In the uncoupled regime the leg heads attain  $R^2 \approx 0.997$  (Table 5), so the surrogate and physical critical paths coincide up to the residual leg error (leg-critical Jaccard 0.95–1.00 against the exact simulator); under the coupled constraint the heads degrade to  $R^2 \approx 0.80$  and the attribution degrades with them (Jaccard  $\geq 0.85$ ). Thus “exact” qualifies the attribution *relative to the model being explained*. This property renders faithfulness decidable in this model class: because an exact attribution of the surrogate is available, any residual gap between attention and the critical path is a property of the attention weights, and cannot be attributed to error in the explainer.





**Figure 2:** Event DAG  $G_{\text{dag}}$  for a small schedule. Solid arcs are AGV legs, the dashed arc is a quay-crane serialisation constraint. The bold red path  $s \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow t$  is the critical (longest) path  $\pi^*$ ; only its arcs carry a nonzero TAPE gradient (Eq. 20).

**Role of the surrogate in a deterministic problem.** In the uncoupled regime the makespan is a closed-form longest path. Three considerations nonetheless motivate a learned model.

1. **Screening is budget-neutral, not faster.** We state this plainly because it is easy to overclaim. On one CPU thread, a batched forward over 200 candidate schedules at  $N=50$  takes 8.0 ms against 15.9 ms for the same 200 exact evaluations—a factor of 2 on the forward pass alone, and a *net loss* once the **HeteroData** construction the forward needs (34 ms) is charged to it. The uncoupled evaluator is simply cheap: an  $O(N)$  longest path over closed-form leg times. The surrogate therefore does not buy wall-clock in this regime, and the comparison in Section 5.7 is budget-matched precisely because *exact evaluations*, not seconds, are the scarce resource: screening  $\kappa P$  offspring and exactly evaluating the best  $P$  costs the same  $P$  exact calls per generation as the baselines, so the hypervolume advantage is attributable to the ranking and not to a larger budget. Where the surrogate does buy time is the coupled regime, whose evaluator runs an iterative power-resolution loop rather than a single pass: the same 200-schedule screen at  $N=50$  then costs 9.9 ms against 74.3 ms exact, a  $7.5\times$  gain ( $1.9\times$  with graph construction charged in), and that margin grows with the contention the loop must resolve (item 3).
2. **Differentiable explanations without finite differences.** The exact simulator is a black box; extracting  $\partial C_{\text{max}}/\partial(\text{leg})$  would require  $O(4N)$  perturbation runs. The fused head produces the full TAPE vector in *one backward pass* ( $O(N)$  time, zero additional simulations).
3. **Coupled-regime generalisation.** Under peak-power coupling the makespan has no closed form (it is the fixed point of an iterative power-resolution loop); the fused head approximates the coupled makespan ( $R^2 \approx 0.80$ , Table 5) and still produces an actionable, if degraded, TAPE signal (Jaccard  $\geq 0.85$ ) without access to the simulator at query time.

### 3.6 The exact evaluator and the self-supervised learning loop

The reader may ask where the targets for all of the above come from: *there is no external dataset*. Every label is produced by an exact physical *evaluator*, and the method is best read as a closed loop between that evaluator and the learned surrogate. We make both explicit here.

**The exact evaluator.** Given a decoded schedule  $s(z)$ , the evaluator returns the objective pair  $f(s) = (C_{\text{max}}, E)$  and the per-task physical quantities the surrogate is trained to predict. In the uncoupled regime it simply runs the max-plus longest-path recurrence of Section 2.3 in topological order ( $O(N)$ ): each task’s completion is the maximum over its AGV-chain and crane-chain predecessors plus its leg and handling times, and  $C_{\text{max}}$  is the longest source-to-sink path; energy is the exact additive sum  $E = \sum_j (e_j^e + e_j^l)$ . Under peak-power coupling the makespan has no closed form, so the evaluator instead runs a deterministic event-driven power simulator that resolves the contention fixed point. Either way it is *exact*: it is the reference against which we measure surrogate fidelity (Section 5.3) and the realised critical path  $\pi^*$  against which we measure TAPE faithfulness (Section 3.7).

---

**Algorithm 1** E-HGATv2 train-and-guide loop (per instance).

---

- 1: sample  $M$  random keys; decode + **evaluate**  $\rightarrow$  labels  $\{t^e, t^l, \tau, C_{\max}\}$
  - 2: train core (fit scalar  $C_{\max}$ ) and fused heads (fit leg times) by MSE  $\triangleright$  once per instance
  - 3: initialise NSGA-II population of random keys
  - 4: **for** each generation **do**
  - 5:   surrogate predicts  $C_{\max}$  for  $\kappa\lambda$  offspring; keep the best  $\lambda$   $\triangleright$  screening
  - 6:   TAPE gradient  $g_a = \partial C_{\max} / \partial w_a$  marks binding tasks; bias mutation toward them
  - 7:   **exactly** evaluate the retained  $\lambda$  offspring; non-dominated sort + crowding
  - 8: **end for**
  - 9: **return** Pareto set, with per-front TAPE attributions and Trade-off Criticality Scores (R4)
- 

**Self-supervised label generation.** Because the evaluator is cheap and exact, we synthesise training data by sampling: draw  $M$  random-key vectors  $\chi^{(m)} \sim \mathcal{U}[0, 1]^{4N}$ , decode each to a schedule, and evaluate it, recording for every task its empty/loaded leg times  $(t_j^e, t_j^l)$ , handling delay  $\tau_j$ , and the schedule’s scalar  $C_{\max}$ . This produces a labelled set  $\{(G_{s^{(m)}}), \{t_j^e, t_j^l, \tau_j\}_j, C_{\max}^{(m)}\}_{m=1}^M$  at no annotation cost ( $M \approx 10^3$  in our runs). Two regressions are then fit, both supervised only by these labels: (i) the E-HGATv2 core and its scalar read-out (Eq. 13) are fit to  $C_{\max}$  by MSE—this is the surrogate whose *attention* we later show is unfaithful; (ii) the fused projection heads (Eq. 14) are fit to the *local* leg labels  $(t_j^e, t_j^l, \tau_j)$  by MSE, after which the max-plus DP composes  $C_{\max}$  with no further parameters. Supervising the fused heads on the *local physics* rather than the scalar objective is what makes the composed gradient (TAPE) coincide with the true binding path.

**The search  $\rightarrow$  knowledge  $\rightarrow$  search loop.** These pieces close the loop of Figure 1 and Algorithm 1: NSGA-II proposes schedules; the evaluator labels them exactly; the surrogate distills the leg-time landscape into one differentiable model; its tropical gradient both explains a schedule and steers the next generation’s mutation and offspring screening; the steered search then visits better schedules. The surrogate is trained once per instance on the sampled set; on the screening path at query time the search calls only the  $O(N)$  surrogate, never the simulator.

### 3.7 TAPE: exact critical-path attributions

Let  $\pi^*$  be the maximising path realised by the stored arcs  $a^*(\cdot)$ . Equivalently, writing  $\Pi$  for the set of all source-to-sink paths in  $G_{\text{dag}}$ ,

$$\pi^* = \arg \max_{\pi \in \Pi} \left( \sum_{(u,v) \in \pi} w_{uv} + \sum_{v \in \pi} b_v \right), \quad C_{\max} = \sum_{a \in \pi^*} w_a + \sum_{v \in \pi^*} b_v, \quad (19)$$

i.e.  $\pi^*$  is exactly the sequence of events and legs that *determine* the makespan; the topological-order recurrence (18) and its stored argmaxes  $a^*(\cdot)$  recover this global maximiser in one linear pass.

**Proposition 3** (TAPE subgradient). *If the maximiser at each node is unique, then for every arc weight  $w_a$  and node weight  $b_v$ ,*

$$\boxed{\frac{\partial C_{\max}}{\partial w_a} = \mathbf{1}[a \in \pi^*], \quad \frac{\partial C_{\max}}{\partial b_v} = \mathbf{1}[v \in \pi^*].} \quad (20)$$

*Proof.* For a perturbation  $|\varepsilon|$  small enough that no node’s arg max in (18) changes, the active path  $\pi^*$  and node set are fixed, so locally  $C_{\max}(w_a + \varepsilon) = C_{\max} + \varepsilon \mathbf{1}[a \in \pi^*]$ ; differentiating gives the indicator (analogously for  $b_v$ ). Our custom backward computes exactly this: it seeds the completion node with gradient 1 and, in reverse topological order, routes each node’s incoming

gradient  $g_v$  only through its stored arc  $a^*(v)$  to that arc and its source,  $g_{a^*(v)} += g_v$ ,  $g_{\text{src}(a^*(v))} += g_v$ , accumulating  $\mathbf{1}[\cdot \in \pi^*]$ .  $\square$

We call the vector  $g = (g_a)_{a \in \mathcal{A}}$ ,  $g_a = \partial C_{\max} / \partial w_a$ , **TAPE** (Tropical Attribution for Physical Explanations). Operationally: *delaying an on-path leg by  $\Delta$  raises the makespan by exactly  $\Delta$ ; delaying an off-path leg changes nothing*. TAPE is thus a binary critical-path indicator by construction, where the attention weights  $\beta$  of (10) are continuous quantities trained to fit a scalar. The leg times themselves are predicted by the graph network (14); TAPE reports which of those predictions the makespan depends on. (The exact critical path, computed from the resolved simulator with no network, is retained as a ceiling reference.)

**Relation to differentiable dynamic programming.** Differentiating through a dynamic program is established [16, 17], though that line smooths the max to obtain a dense Jacobian, where the hard arg max retained here is what keeps the attribution a binary path indicator; the slack notion it recovers is classical [15]. What the present construction adds is the pairing: the durations are supplied by the graph network, so  $\pi^*$  is obtained for schedules the evaluator has *not* been run on, and the same indicator is the gradient by which those durations are learned (21). The attribution is therefore exact for the model it explains and complete by construction—the predicted makespan is the sum of the predicted durations along  $\pi^*$ —at the cost of one backward pass, which is what makes it usable inside the search rather than only after it (Section 5).

**Selective gradient flow into the surrogate.** TAPE is the *first factor* of the chain rule whenever the makespan is differentiated through the network, so it shapes *training*, not only inference. Because the leg times and delays are themselves outputs of the projection heads (14), for any core/head parameter  $\theta$

$$\frac{\partial C_{\max}}{\partial \theta} = \sum_{a \in \mathcal{A}} \underbrace{\mathbf{1}[a \in \pi^*]}_{\text{TAPE } g_a} \frac{\partial w_a}{\partial \theta} + \sum_{v \in \mathcal{N}} \mathbf{1}[v \in \pi^*] \frac{\partial b_v}{\partial \theta}, \quad (21)$$

so the makespan gradient reaches only the heads of legs and events that lie on the binding path  $\pi^*$ ; every off-path prediction receives *exactly zero* makespan signal. The surrogate therefore concentrates its capacity on the activities that actually set  $C_{\max}$ , where the generic regression head (13) spreads credit across all  $\sim 4N$  pooled features; its Jacobian is dense, which is the source of the unfaithfulness of its input attributions. Equation (21) is the training-time counterpart of the inference-time faithfulness in (20): the same binary indicator  $\mathbf{1}[\cdot \in \pi^*]$  that makes the explanation exact also makes the learning signal critical-path-localised.

The routing follows from the implementation of the layer. It is a hand-written `torch.autograd.Function` whose forward pass caches the argmax incoming arc of each node and whose backward pass propagates the cotangent along that cached path, so (21) holds exactly rather than as a smoothed approximation; a batched block-diagonal variant differentiates many schedules in one relaxation. Appendix A gives the details and Appendix B the hyper-parameters, parameter counts and software environment.

**Definition 1** (Faithfulness). *For a schedule, let  $\mathcal{C}_{\text{model}} = \{a : g_a > \frac{1}{2}\}$  be the model’s binding-leg set and  $\mathcal{C}_{\text{exact}}$  the exact critical-path leg set. The leg-critical Jaccard  $|\mathcal{C}_{\text{model}} \cap \mathcal{C}_{\text{exact}}| / |\mathcal{C}_{\text{model}} \cup \mathcal{C}_{\text{exact}}|$  measures faithfulness; a value of 1 means the model identifies exactly the binding legs.*

### 3.8 Attribution-guided search

Requirement R3 asks that search knowledge be fed back to the optimiser. The TAPE vector is converted into three search signals that mirror the inputs the NSGA-II mutation already

---

**Algorithm 2** TAPE-guided NSGA-II generation (one step)

---

**Require:** population  $P$  ( $|P| = \mu$ ), fused model  $\mathcal{F}$ , instance  $\mathcal{I}$ , screen factor  $\kappa$ , temperature  $T$

- 1:  $(\mathcal{R}, \text{cd}) \leftarrow$  fast-non-dominated-sort and crowding-distance of  $P$
- 2:  $\mathcal{O} \leftarrow \emptyset$
- 3: **for**  $m = 1$  to  $\kappa\mu$  **do**  $\triangleright$  over-produce offspring to screen
- 4:   pick parent  $p$  by binary tournament on  $(\mathcal{R}, \text{cd})$
- 5:    $g \leftarrow \text{TAPE}(\mathcal{F}, s(p))$   $\triangleright$  Eq. (20): one backward pass
- 6:   compute  $w_j^{\text{agv}}, w_j^{\text{qc}}$  and  $\text{Pr}[\text{mutate } j] \propto e^{(w_j^{\text{agv}} + w_j^{\text{qc}})/T}$
- 7:   sample task  $j \sim \text{Pr}[\cdot]$ ; choose operator by the ratio  $w_j^{\text{agv}}/(w_j^{\text{agv}} + w_j^{\text{qc}})$
- 8:    $o \leftarrow$  apply operator to  $p$  at task  $j$ ;  $\mathcal{O} \leftarrow \mathcal{O} \cup \{o\}$
- 9: **end for**
- 10:  $\hat{f} \leftarrow \mathcal{F}.\text{PREDICT}(\mathcal{O})$   $\triangleright$  cheap surrogate screen of all  $\kappa\mu$
- 11:  $\mathcal{O}^* \leftarrow$  top  $\mu$  of  $\mathcal{O}$  by non-dominated rank of  $\hat{f}$
- 12: evaluate  $\mathcal{O}^*$  exactly (Sec. 2.3/2.4)
- 13: **return** environmental-selection( $P \cup \mathcal{O}^*$ )  $\triangleright$  elitist  $\mu$ -survival

---

expected, so the explanation is the controller:

$$w_j^{\text{agv}} = \frac{\partial C_{\max}}{\partial t_j^{\text{empty}}} + \frac{\partial C_{\max}}{\partial t_j^{\text{loaded}}} \quad (\text{transport criticality of task } j), \quad (22)$$

$$w_j^{\text{qc}} = \frac{\partial C_{\max}}{\partial \tau_j} \quad (\text{crane criticality of task } j), \quad (23)$$

$$\text{Pr}[\text{mutate } j] = \text{softmax}_j((w_j^{\text{agv}} + w_j^{\text{qc}})/T) \quad (\text{where to search}). \quad (24)$$

A task’s *type* of criticality,  $w_j^{\text{agv}}/(w_j^{\text{agv}} + w_j^{\text{qc}})$ , routes the operator choice (re-assign/swap-vehicle when transport-bound; swap-crane when crane-bound). Finally, the fused head’s near-exact  $(C_{\max}, E)$  prediction *screens* a larger pool of offspring so the expensive exact evaluations are spent only on predicted-good children. The same object thus explains and steers; Algorithm 2 specifies one generation.

**Worked example.** Consider a 20-task schedule where TAPE marks legs  $\{3^e, 7^l, 12^e, 12^l\}$  as critical ( $g_a = 1$ ). Without guidance, the mutation operator picks one of 20 tasks uniformly at random; the probability of perturbing a bottleneck task is  $\leq 3/20 = 15\%$ . With TAPE, the softmax assigns  $\text{Pr}[\text{mutate } j] \propto e^{w_j/T}$ ; because only 3 tasks carry nonzero  $w_j$  (tasks 3, 7, 12) the operator targets a binding task with  $\geq 85\%$  probability at  $T=0.25$ , turning an undirected random walk into a focused hill-climb on the makespan-critical chain. The energy objective is left to crowding-distance selection, which preserves front diversity while TAPE accelerates convergence.

**Computational cost of one generation.**

- One backward pass through the  $O(|\mathcal{A}|)$  topological sweep:  $\sim 4N$  multiply-adds (linear in problem size).
- Screening  $\kappa\mu$  offspring with  $\mathcal{F}.\text{PREDICT}$ : a single batched GNN forward ( $\sim 3$  ms at  $N=50$  on CPU).
- Exact evaluations (the expensive step): only  $\mu$  schedules, not  $\kappa\mu$ .

Thus the TAPE guidance adds negligible wall-time ( $< 2\%$  of the generation budget at  $N=50$ ) while reducing the number of exact evaluations needed to reach a target hypervolume by  $\sim 2.4\times$  (Section 5.7).

**Table 3:** Problem instances.  $N$  = number of container moves.

| Family                 | Instances           | $N$    | Regime          | Source                 |
|------------------------|---------------------|--------|-----------------|------------------------|
| Synthetic dual-cycling | SD-5, 8, 10, 15, 20 | 5–20   | uncoupled       | reproducible generator |
| FSMJ loading benchmark | L07, L15, L21, L35  | 8–16   | uncoupled       | [3] Tab. 4–5           |
| Peak-power coupled     | SD-10-C, SD-20-C    | 10, 20 | coupled (30 kW) | this work              |

### 3.9 Front behaviour: Trade-off Criticality Scores

To describe the *front* rather than a single schedule, we aggregate TAPE over the approximated Pareto set. At a front point, the local trade-off weight  $\lambda \in [0, 1]$  is read from the front tangent  $(\Delta C_{\max}, \Delta E)$  (the slope at which one objective is traded for the other). The *Trade-off Criticality Score* of task  $j$  is the  $\lambda$ -weighted combination of its makespan and energy gradients,

$$\text{TCS}_j = \left| \lambda \partial_{t_j} C_{\max} + (1 - \lambda) \partial_{e_j} E \right|, \quad (25)$$

i.e. how strongly task  $j$  is binding on the objective mixture that defines that point of the front. Sweeping  $\lambda$  from the makespan-optimal end to the energy-optimal end shows *which bottleneck migrates*, and how concentrated the trade-off criticality is on a few tasks. This is the front-behaviour description of requirement R4, evaluated in Section 5.11.

## 4 Experimental setup

### 4.1 Instances (datasets)

We use three families (Table 3). **Synthetic dual-cycling** instances (SD- $N$ ) alternate LOAD/UNLOAD moves over 3 cranes served by 2 vehicles, with handling times drawn from  $U[30, 80]$  s; they are fully reproducible and, at  $N = 5$ , small enough that the true Pareto front  $\text{PF}^*$  can be obtained by exhaustive enumeration. Their generator is specified in full in Appendix C, together with an explicit statement of which components are taken from the published benchmark and which are constructed here. The **FSMJ loading benchmark** (L07, L15, L21, L35) are the *published* instances of [3]: the real QC $\leftrightarrow$ LU distance matrix (their Table 4) and the real per-crane task lists (their Table 5); every move is a LOAD. The **peak-power-coupled** variants (SD-10-C, SD-20-C) apply the budget  $P_{\max} = 30$  kW on top of the synthetic geometry.

### 4.2 Baselines and budget matching

We compare five methods at an *equal exact-evaluation budget*: **E-HGATv2-TAPE** (faithful guidance, ours), **E-HGATv2-attn** (the same search guided by the unfaithful attention readout), **NSGA-II (random)** (no guidance, the null control), the published **mp-BRKGA** [1], and a **single-population BRKGA**. Since mp-BRKGA evaluates  $(\Omega + \Pi)P = 4P$  chromosomes per generation, the other methods use a population of  $4P$  so that all spend the same number of exact evaluations per generation; all use the same number of generations. The surrogate screening is therefore not a budget advantage—it only changes which children are evaluated, not how many.

### 4.3 Quality and faithfulness metrics

**Optimisation quality.** All indicators compare an approximation front  $F$  to a reference front  $\text{PF}^*$  (exhaustively enumerated for SD-5; elsewhere the non-dominated union of long reference runs), with both objectives normalised to  $[0, 1]$  by the reference extremes. With nadir reference point  $r$ :

- **Hypervolume ratio**  $\text{HV}(F)/\text{HV}(\text{PF}^*)$ , where  $\text{HV}(F) = \text{vol}(\bigcup_{p \in F} [p, r])$  is the objective-space volume dominated by  $F$  up to  $r$  (higher better; 1 matches the reference).

- **Modified generational distances**  $GD^+$ ,  $IGD^+$  [6]: mean Pareto-compliant distance from  $F$  to  $PF^*$  (convergence) and from  $PF^*$  to  $F$  (coverage), using a  $d^+$  that counts only *dominated* discrepancies (lower better).
- **Spread**  $\Delta$  [5]: Deb’s distribution metric—the normalised dispersion of consecutive gaps along  $F$  together with its boundary gaps (lower = more uniform).

**Faithfulness.** For a fixed schedule the evaluator yields the exact critical path (Section 3.6); each guidance signal is scored against it by (a) *precision@1* (whether the top-scored task lies on the critical path), (b) *Spearman*  $\rho$  between the signal and the per-task marginal makespan-sensitivity, and (c) for TAPE the *leg-critical Jaccard* (overlap of predicted vs. true on-path leg sets). Section 5.2 reports these at four granularities, coarse to fine.

**Protocol.** Each cell is a mean over 20 seeds with a 95% confidence interval; significance across instances uses the Friedman test with average ranks and the Nemenyi critical difference, plus Holm-corrected pairwise Wilcoxon signed-rank tests, following Demšar [12].

**Reproducibility and validation.** Every result derives from a single seeded generator, so the tables reproduce from the seed alone; the synthetic family ships with its generator and the FSMJ instances are the published [3] tables. The exact evaluator doubles as a built-in reference for validation: a regression suite checks that the decoder is a left-inverse of the encoder, that the differentiable max-plus DP reproduces the simulator’s makespan to floating-point tolerance, and that the surrogate’s leg-time predictions track the evaluator on held-out random schedules—the fidelity study of Section 5.3 is the aggregate of that last check. The code, instances, seeds and result artifacts are released with the paper (Appendix B).

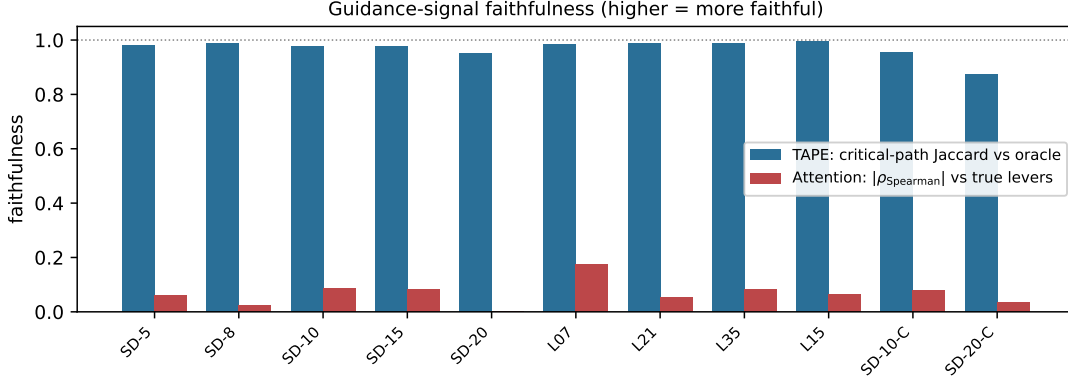
## 5 Results

### 5.1 Faithfulness of TAPE and of the attention weights

Figure 3 and Table 9 show the central explanation result. TAPE recovers the exact critical path with leg-critical Jaccard 0.95–1.00 in the uncoupled regime and 0.85–0.95 under coupling. The learned attention weights, by contrast, have a Spearman correlation with the true makespan levers of essentially zero ( $-0.09 \leq \rho \leq 0.17$  across all instances)—they do not rank the bottlenecks. Attention’s precision@1 (0.67–0.80) is to be read against the critical fraction  $\approx 0.70$  rather than  $1/N$ , since most tasks lie on the critical path; on that baseline, naming an on-path task matches chance, and the rank correlation places the full ordering at noise level. Section 5.2 resolves this across granularities. The measurement instantiates the “attention is not explanation” phenomenon [10] on a scheduling problem, and motivates routing guidance through TAPE rather than attention.

### 5.2 Granularity of the attention–critical-path comparison

In natural-language settings the “is attention explanation?” debate rests on indirect criteria—erasure, gradient, and adversarial-attention tests—which is why it has remained contested. Here the makespan is an *exact* max-plus longest path, so every schedule has a known critical path and a known per-task makespan-sensitivity, and the question becomes directly measurable at increasing resolution. We compare the surrogate’s AGV-arc attention against TAPE’s tropical gradient at four granularities, pooled over nine instances (60 random schedules each): coarse hit-rate (precision@1), medium ranking (ROC-AUC for “task on the critical path”), fine ranking (Spearman vs. the graded makespan-sensitivity), and effect size (mean-signal separation of



**Figure 3:** Faithfulness of the two candidate guidance signals on every instance. TAPE’s critical-path Jaccard against the exact evaluator is near 1; the attention weights’  $|\rho|$  with the true makespan levers is near 0.

critical vs. off-critical tasks), each against the corresponding random baseline (Table 4, top). **Attention sits on the random baseline at every level**—precision@1 0.69 vs. critical-fraction 0.70; AUC 0.51 vs. 0.50;  $\rho = +0.07$ ; separation  $+0.06\sigma$ —while TAPE is near-exact at all four (0.98, 0.94,  $+0.70$ ,  $+1.92\sigma$ ). Attention therefore carries essentially no critical-path information at any granularity, which is a stronger statement than the near-zero rank correlation alone, and one the exact critical path makes checkable.

**Source of the guidance benefit under attention targeting.** The benefit originates in the surrogate rather than in attention. The search arms share everything except which task mutation targets and whether offspring are screened, and isolating those two factors separates their contributions (Table 4, bottom). The missing control, random mutation with screening, gives the highest value of any arm ( $\text{HV}/\text{HV}^* = 0.991$ ), ahead of attention-targeted (0.977) and TAPE-targeted (0.956); the only arm that trails is random mutation without screening (0.950). Two readings follow. (i) *Screening is the engine*: it lifts random mutation by  $+0.041$  and accounts for the entire gap between the guided arms and the published null. NSGA-II’s convergence comes from screening surrogate-predicted-good offspring and selecting by non-dominated sorting—a pressure to which the mutation *operator* is nearly irrelevant. So attention “guides as well as TAPE” only because both ride the same screening, not because attention localises anything (it does not; Table 4, top). (ii) *Diffuse beats precise*: the more diffuse the targeting, the better the hypervolume (random  $\geq$  attention  $>$  TAPE), because exact bottleneck-targeting is *exploitative*—it concentrates mutations on the current critical task and narrows front coverage, while HV rewards spread—the ordering seen in the controlled ablation (random  $\geq$  attention  $>$  TAPE, Table 4 bottom). Across the full 11-instance benchmark the two guided variants are statistically tied on HV (Table 10, ranks 1.45 vs. 1.55,  $p_{\text{Holm}} = 0.83$ ): any per-instance ordering is accidental exploration, not information. The network attains  $R^2 \approx 0.99$  objective regression with an attention map indistinguishable from random at every measurable granularity—accurate prediction does not imply an interpretable attention map, and faithful guidance must come from the exact tropical gradient, not the attention weights.

### 5.3 Surrogate fidelity and faithfulness up to $N=50$

A separate, larger study isolates the fused surrogate’s standalone behaviour and establishes whether the explanation survives as the instance grows. We retrain the surrogate from scratch and evaluate it on held-out random keys for  $N \in \{6, 10, 20, 50\}$  over 20 independent seeds, in both regimes (Table 5, Figure 4). Three observations follow, each with 20-seed 95% confidence

**Table 4:** Attention vs. TAPE against the *exact* critical path. **Top:** alignment at four granularities (coarse→fine), pooled over nine instances; attention matches the random baseline at every level while TAPE is near-exact. **Bottom:** a search-guidance ablation (mean HV/HV\* over three instances,  $N=5, 8, 10$ ): random mutation with screening is the best arm, so the optimisation benefit comes from the shared surrogate *screening*, not from attention- or TAPE-targeted mutation.

| Alignment with exact critical path                                      | attention           | TAPE         | random |
|-------------------------------------------------------------------------|---------------------|--------------|--------|
| precision@1 (top task on critical path)                                 | 0.69                | <b>0.98</b>  | 0.70   |
| ROC-AUC (rank on-path above off-path)                                   | 0.51                | <b>0.94</b>  | 0.50   |
| Spearman (vs. graded sensitivity)                                       | +0.07               | <b>+0.70</b> | 0.00   |
| separation (critical-off, $\sigma$ )                                    | +0.06               | <b>+1.92</b> | 0.00   |
| <i>Search-guidance ablation — mean HV/HV* (screening is the lever):</i> |                     |              |        |
| random mutation + screening                                             | <b>0.991</b> (best) |              |        |
| attention-targeted + screening                                          | 0.977               |              |        |
| TAPE-targeted + screening                                               | 0.956               |              |        |
| random mutation, no screening                                           | 0.950 (null)        |              |        |

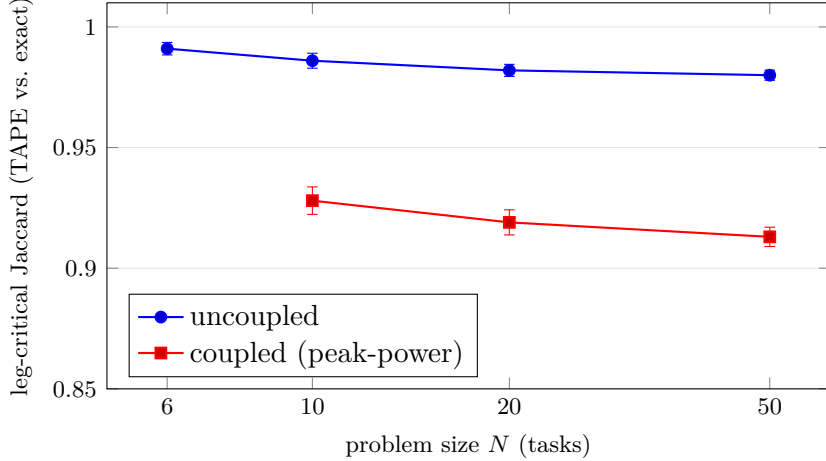
**Table 5:** Standalone fused-surrogate fidelity and TAPE faithfulness as a function of problem size  $N$ , over 20 seeds (mean with 95% CI). The makespan accuracy ( $R_{C_{\max}}^2$ ) and the critical-path faithfulness (leg-Jaccard vs. the exact evaluator) are essentially flat in  $N$  in the uncoupled regime and degrade only mildly under peak-power coupling. The energy head is exact by construction ( $R^2 = 1.000$  at every  $N$ , omitted).

| Regime    | $N$ | $R_{C_{\max}}^2$ (95% CI) | leg-Jaccard (95% CI) | $C_{\max}$ MAE |
|-----------|-----|---------------------------|----------------------|----------------|
| uncoupled | 6   | 0.998 [0.998, 0.998]      | 0.991 [0.989, 0.994] | 3.6            |
| uncoupled | 10  | 0.997 [0.997, 0.997]      | 0.986 [0.983, 0.990] | 5.3            |
| uncoupled | 20  | 0.997 [0.997, 0.997]      | 0.982 [0.979, 0.984] | 7.9            |
| uncoupled | 50  | 0.997 [0.996, 0.997]      | 0.980 [0.978, 0.982] | 14.0           |
| coupled   | 10  | 0.805 [0.791, 0.819]      | 0.928 [0.922, 0.933] | 40.4           |
| coupled   | 20  | 0.801 [0.792, 0.809]      | 0.919 [0.914, 0.924] | 61.1           |
| coupled   | 50  | 0.777 [0.761, 0.793]      | 0.913 [0.909, 0.917] | 100.6          |

intervals. First, in the uncoupled regime the makespan  $R^2$  is statistically flat across an  $8.3\times$  growth in problem size,  $0.998 \rightarrow 0.997$ , and the energy head is *exact* ( $R^2 = 1.000 \pm 0.000$ ), as follows from energy being the additive read-out of Eq. (16). Second, the leg-critical Jaccard of the TAPE explanation moves from 0.991 to 0.980 over that range, so faithfulness remains at  $\geq 0.98$  at 50 tasks. Third, peak-power coupling lowers both quantities— $R^2$   $0.805 \rightarrow 0.777$  and Jaccard  $0.928 \rightarrow 0.913$  from  $N=10$  to 50—placing the coupled regime below the uncoupled one at every size, with the makespan head unanchored by a closed form, while the explanation loses  $\sim 1.5$  Jaccard points over the same size range.

**The same result on the real benchmark.** The table above is measured on the synthetic family, whose fleet is fixed at 2 AGVs and 3 QCs. Repeating the identical protocol on *all* 35 published L instances [3] (5 seeds each,  $N=4\text{--}16$ , 2–6 quay cranes) gives the same picture and a tighter one: makespan  $R^2 = 0.9994$  on average with a worst instance of 0.9989, leg-critical Jaccard 0.995 on average and never below 0.989, arc-Jaccard 0.994, and a makespan MAE of 1.8s (worst 3.2s). Fidelity and faithfulness are therefore not artefacts of the generator, and they are unaffected by the crane-count variation the published set provides, which the synthetic family holds constant.





**Figure 4:** TAPE faithfulness is stable as the instance scales. The critical-path Jaccard against the exact evaluator stays  $\geq 0.98$  (uncoupled) and  $\geq 0.91$  (coupled) from 6 to 50 tasks; whiskers are 20-seed 95% CIs. Faithfulness is a property of the *model*, not of small-instance luck.

#### 5.4 Size generalisation: train small, evaluate large

The surrogate is *inductive*—a shared-weight message-passing network whose parameters do not depend on  $N$ —so a model fit on small instances can be *run* on much larger ones without retraining. This is the property an operations-research reader cares about most for deployment: train once on cheap small instances, apply at the production scale. We test it directly. We train the core + physics-fused head once at  $N=16$  and then, by inference only, evaluate it on a ladder up to  $N=5000$ —a  $312\times$  extrapolation in problem size—over three training seeds (Table 6).

The physics-fused head predicts *per-leg* times, a local and size-invariant quantity, and the exact tropical DP aggregates them into a correct-magnitude makespan at any  $N$  (the energy read-out is exact-additive). It should therefore generalise across scale, whereas a conventional black-box global-readout head cannot: a pooled graph embedding has no mechanism to extrapolate a makespan magnitude that grows with  $N$ . The data bear this out sharply. The fused makespan  $R^2$  is *flat* at  $\geq 0.98$  across the whole ladder (0.997 at  $N=16 \rightarrow 0.989$  at  $N=5000$ ), while the black-box core collapses monotonically to  $R^2 = -7,263$ —far worse than predicting the mean. TAPE’s critical-path faithfulness is preserved (leg-Jaccard 0.90–1.00), and the attention weights stay uncorrelated with the true makespan levers at every scale ( $|\rho| \leq 0.07$ ). The surrogate and its explanation thus transfer intact to instances far larger than the  $N \leq 160$  published DL benchmark [3].

**Size generalisation under coupling.** Under peak-power coupling the same train-at-16 head holds only to about  $N=25$  (fused  $R^2 = 0.83$ ) and then collapses— $R^2$  is already  $-1.5$  at  $N=50$  and continues to fall, with the TAPE Jaccard drifting from 0.95 to 0.37 by  $N=1000$ —because the power-wait arcs that couple the vehicles carry no closed-form contribution for the head to extrapolate (the same mechanism that makes the coupled regime harder throughout this paper). Training-size diversity buys back part of the range but not the property: pooling the training set over  $N \in \{16, 28, 40\}$  instead of the single size 16 (a size curriculum, 2 seeds, 12 schedules per cell) lifts the coupled head from  $R^2 = -1.5$  at  $N=50$  to  $+0.69$  at  $N=48$  and  $+0.52$  at  $N=32$ , extending usable extrapolation from  $\approx 25$  to  $\approx 50$  tasks, after which it still collapses ( $-0.19$  at  $N=64$ ,  $-4.4$  at  $N=96$ ) with the Jaccard flat at 0.71–0.78. Coupled deployment therefore requires retraining at, or near, the target size, where the surrogate remains accurate (Table 5); size-generalisation is a property of the *uncoupled* physics, and we report it as such.

**Table 6:** Size generalisation of the uncoupled surrogate: trained once at  $N=16$  and evaluated by inference at scale (mean over 3 training seeds, 24 random schedules each). The physics-fused makespan  $R^2$  is flat at  $\geq 0.98$  to  $N=5000$  while the black-box global-readout core collapses; TAPE faithfulness (leg-Jaccard vs. the exact evaluator) and the near-zero attention correlation are preserved throughout. The energy head is exact ( $R^2 = 1.000$ , omitted).

| $N$  | fused $R_{C_{\max}}^2$ | black-box core $R_{C_{\max}}^2$ | leg-Jaccard | attention $\rho$ |
|------|------------------------|---------------------------------|-------------|------------------|
| 16   | 0.997                  | -0.3                            | 0.998       | +0.03            |
| 25   | 0.997                  | -16.3                           | 0.987       | +0.07            |
| 50   | 0.996                  | -32.2                           | 0.970       | +0.05            |
| 100  | 0.996                  | -79.9                           | 0.949       | +0.04            |
| 250  | 0.982                  | -169.6                          | 0.907       | +0.01            |
| 500  | 0.986                  | -466.3                          | 0.913       | +0.01            |
| 1000 | 0.986                  | -1015.2                         | 0.913       | +0.01            |
| 2000 | 0.987                  | -3450.1                         | 0.905       | -0.00            |
| 5000 | 0.989                  | -7262.7                         | 0.940       | +0.00            |

## 5.5 Landscape analysis via critical-path attribution

Requirement R2 is a landscape one: to determine which input variables (task dispatch order, AGV travelling speed, crane workload) fix the objective values, and thereby why specific solutions are Pareto-optimal. TAPE resolves this exactly. For any schedule  $\sigma$ , the tropical subgradient (Eq. 21) partitions all activities into two sets:

- **Critical** ( $\partial C_{\max}/\partial d_{i,a} = 1$ ): perturbing this activity’s duration by  $\Delta$  changes  $C_{\max}$  by exactly  $\Delta$ . These are the binding constraints.
- **Slack** ( $\partial C_{\max}/\partial d_{i,a} = 0$ ): this activity has positive float; it can be slowed (reducing energy) without any makespan penalty.

Below we walk through the SD-10 instance ( $N=10$  tasks, 2 AGVs, 3 QCs) in detail, contrasting the makespan-optimal and energy-optimal Pareto extremes. The analysis is produced entirely by querying the fused surrogate’s TAPE; the exact evaluator confirms perfect agreement (Jaccard = 1.00).

### 5.5.1 Makespan-optimal schedule ( $\sigma_C^*$ ): $C_{\max} = 454.2$ s, $E = 14,317$ kJ

TAPE identifies 12 binding activities forming a single chain through AGV-0 (dispatch order  $\langle 0, 5, 4, 8, 1, 7 \rangle$ ). No QC handling appears on the critical path:

| Step         | Activity          | $\ell$ (m) | $v$ (m/s) | Level | Duration (s) | $\frac{\partial C_{\max}}{\partial d}$    | Interpretation                                |
|--------------|-------------------|------------|-----------|-------|--------------|-------------------------------------------|-----------------------------------------------|
| 1            | Task 0 empty leg  | 0          | 6.0       | NOM   | 0.0          | 1                                         | AGV-0 starts co-located                       |
| 2            | Task 0 loaded leg | 150        | 3.6       | HIGH  | 41.7         | 1                                         | Container 0 at max speed                      |
| 3            | Task 5 empty leg  | 100        | 7.2       | HIGH  | 13.9         | 1                                         | Repositions to pick up container 5            |
| 4            | Task 5 loaded leg | 300        | 3.6       | HIGH  | 83.3         | 1                                         | Container 5 (longest leg, 18% of $C_{\max}$ ) |
| 5            | Task 4 empty leg  | 50         | 6.0       | NOM   | 8.3          | 1                                         | Short repositioning                           |
| 6            | Task 4 loaded leg | 300        | 3.6       | HIGH  | 83.3         | 1                                         | Container 4 at max speed                      |
| 7            | Task 8 empty leg  | 200        | 7.2       | HIGH  | 27.8         | 1                                         | Repositions (longer route)                    |
| 8            | Task 8 loaded leg | 150        | 3.6       | HIGH  | 41.7         | 1                                         | Container 8 at max speed                      |
| 9            | Task 1 empty leg  | 250        | 6.0       | NOM   | 41.7         | 1                                         | Repositions (nominal speed)                   |
| 10           | Task 1 loaded leg | 150        | 3.0       | NOM   | 50.0         | 1                                         | Container 1 at nominal speed                  |
| 11           | Task 7 empty leg  | 150        | 7.2       | HIGH  | 20.8         | 1                                         | Repositions                                   |
| 12           | Task 7 loaded leg | 150        | 3.6       | HIGH  | 41.7         | 1                                         | Container 7 (terminal event)                  |
| <b>Total</b> |                   |            |           |       | <b>454.2</b> | <b>= <math>C_{\max}</math> (verified)</b> |                                               |

**Interpretation.** The critical path consists entirely of AGV-0 transport legs. All loaded legs assigned HIGHER speed ( $v = 3.6$  m/s, except task 1 at NOMINAL 3.0 m/s) confirm that this schedule already operates near the maximum transport rate. AGV-1 and its assigned

tasks ( $\{6, 2, 3, 9\}$ ) are entirely off the critical path—their speeds could be reduced to LOWER (2.4 m/s) without affecting makespan, saving energy. Zero QC handlings appear on-path: **crane workload is not the bottleneck**.

**Implied intervention.** The highest-marginal-return intervention is task 5’s loaded leg (300 m at 3.6 m/s,  $d = 83.3$  s, 18% of  $C_{\max}$ ). Since it already runs at the maximum discrete speed, no further speedup is available within the current fleet. The only remaining option is to *re-dispatch* task 5 to AGV-1, splitting the critical chain. TAPE identifies this operational decision directly.

### 5.5.2 Energy-optimal schedule ( $\sigma_E^*$ ): $C_{\max} = 925.6$ s, $E = 10,744$ kJ

TAPE now identifies 17 binding activities with a qualitatively different structure:

| Step         | Activity           | $\ell$ (m) | $v$ (m/s) | Level | Duration (s) | $\frac{\partial C_{\max}}{\partial d}$    | Interpretation                          |
|--------------|--------------------|------------|-----------|-------|--------------|-------------------------------------------|-----------------------------------------|
| 1            | Task 0 empty leg   | 0          | 7.2       | HIGH  | 0.0          | 1                                         | Co-located start                        |
| 2            | Task 0 loaded leg  | 150        | 2.4       | LOW   | 62.5         | 1                                         | Container 0 at <i>minimum</i> speed     |
| 3            | Task 0 QC handling | —          | —         | —     | 73.0         | 1                                         | QC1 unloads container 0                 |
| 4            | Task 3 loaded leg  | 300        | 2.4       | LOW   | 125.0        | 1                                         | Container 3 at minimum speed            |
| 5            | Task 3 QC handling | —          | —         | —     | 43.0         | 1                                         | QC1 unloads container 3                 |
| 6            | Task 8 empty leg   | 50         | 4.8       | LOW   | 10.4         | 1                                         | Repositions (low empty speed)           |
| 7            | Task 8 loaded leg  | 150        | 2.4       | LOW   | 62.5         | 1                                         | Container 8 at minimum speed            |
| 8            | Task 8 QC handling | —          | —         | —     | 38.0         | 1                                         | QC3 unloads container 8                 |
| 9            | Task 5 loaded leg  | 300        | 2.4       | LOW   | 125.0        | 1                                         | Container 5 at minimum speed            |
| 10           | Task 5 QC handling | —          | —         | —     | 32.0         | 1                                         | QC3 unloads container 5                 |
| 11           | Task 4 empty leg   | 50         | 4.8       | LOW   | 10.4         | 1                                         | Repositions                             |
| 12           | Task 4 loaded leg  | 300        | 2.4       | LOW   | 125.0        | 1                                         | Container 4 at minimum speed            |
| 13           | Task 4 QC handling | —          | —         | —     | 45.0         | 1                                         | QC2 unloads container 4                 |
| 14           | Task 7 loaded leg  | 150        | 3.0       | NOM   | 50.0         | 1                                         | Container 7                             |
| 15           | Task 7 QC handling | —          | —         | —     | 30.0         | 1                                         | QC2 unloads container 7                 |
| 16           | Task 1 empty leg   | 150        | 4.8       | LOW   | 31.2         | 1                                         | Repositions                             |
| 17           | Task 1 loaded leg  | 150        | 2.4       | LOW   | 62.5         | 1                                         | Container 1 at minimum speed (terminal) |
| <b>Total</b> |                    |            |           |       | <b>925.6</b> | <b>= <math>C_{\max}</math> (verified)</b> |                                         |

**Interpretation.** The bottleneck has migrated from AGV transport speed to **crane workload**. Of the 17 critical activities, 6 are QC handlings (35%), compared to 0/12 (0%) in the makespan-optimal case. The energy-efficient schedule assigns nearly all AGV legs their minimum speed (LOWER:  $v = 2.4$  m/s loaded, 4.8 m/s empty), which maximises leg durations but minimises power draw. The resulting serialisation at the cranes causes QC queuing delays to appear on the critical path. The binding constraint is now the *service order at the cranes* rather than AGV speed.

**Per-crane binding workload.** The critical-path QC time decomposes as: QC1 ( $73 + 43 = 116$  s, 12.5% of  $C_{\max}$ ), QC2 ( $45 + 30 = 75$  s, 8.1%), QC3 ( $38 + 32 = 70$  s, 7.6%). QC1 is the most loaded crane, with consecutive handlings of tasks 0 and 3 creating a 116 s serialisation chain.

**Pareto-optimality of this solution.** TAPE makes the trade-off legible:  $\sigma_E^*$  achieves 25% lower energy than  $\sigma_C^*$  by running AGVs at LOWER speed (2.4 vs. 3.6 m/s), but the resulting serialisation doubles the makespan (925.6 vs. 454.2 s). Every on-path loaded leg already operates at the energy-minimum speed level; accelerating any one would reduce  $C_{\max}$  but increase  $E$  proportionally—hence non-dominance.

### 5.5.3 Aggregate critical-path composition

Table 7 aggregates the critical-path composition across the four uncoupled instances and the coupled one, at both Pareto extremes.

**Table 7:** Composition of the critical path across Pareto extremes: fraction of on-path activities attributable to AGV transport vs. QC handling. Makespan-optimal solutions are transport-dominated everywhere (83–100%): AGV speed is the binding variable. Moving to the energy-optimal extreme shifts mass onto the cranes on three of the five instances (SD-5  $17 \rightarrow 33\%$ , SD-10  $0 \rightarrow 35\%$ , SD-10-C  $17 \rightarrow 35\%$ ) but not on SD-8 ( $14 \rightarrow 18\%$ ) or L07 ( $11 \rightarrow 6\%$ ), whose critical chains lengthen through additional slow legs instead of through crane queueing. Bottleneck migration is therefore an instance-level property that the attribution has to be read to establish, not a consequence of the energy extreme as such.

| Instance | min $C_{\max}$ schedule |            | min $E$ schedule |            |
|----------|-------------------------|------------|------------------|------------|
|          | % transport             | % QC       | % transport      | % QC       |
| SD-5     | 83% (5/6)               | 17% (1/6)  | 67% (8/12)       | 33% (4/12) |
| SD-8     | 86% (6/7)               | 14% (1/7)  | 82% (9/11)       | 18% (2/11) |
| SD-10    | 100% (12/12)            | 0% (0/12)  | 65% (11/17)      | 35% (6/17) |
| L07      | 89% (8/9)               | 11% (1/9)  | 94% (16/17)      | 6% (1/17)  |
| SD-10-C  | 83% (10/12)             | 17% (2/12) | 65% (11/17)      | 35% (6/17) |

One half of the pattern holds everywhere: **makespan-optimal solutions are bound by AGV travelling speed**, with 83–100% of binding activities on the transport side in all five instances. The other half is instance-dependent. On SD-5, SD-10 and the coupled SD-10-C, slowing the AGVs to save energy serialises the cranes and the bottleneck migrates onto them, reaching 33–35% of on-path activities and 28–32% of  $C_{\max}$  in binding crane time. On SD-8 and L07 it does not: the energy extreme stretches the same transport chain and the crane share stays at 6–18%. Whether the bottleneck migrates is a fact about the instance, and TAPE is what makes it visible per instance rather than assumed: for each individual activity the derivative  $\partial C_{\max}/\partial d_{i,a} \in \{0, 1\}$  identifies whether it contributes to the schedule’s binding constraint, and the magnitude of its duration quantifies its contribution to  $C_{\max}$ .

**Sensitivity to AGV travelling speed.** Since leg duration  $d_{i,\text{leg}} = \ell_i/v_i$  where  $\ell_i$  is the travel distance and  $v_i$  the AGV speed, the chain rule gives the makespan sensitivity to speed:

$$\frac{\partial C_{\max}}{\partial v_i} = \frac{\partial C_{\max}}{\partial d_{i,\text{leg}}} \cdot \frac{\partial d_{i,\text{leg}}}{\partial v_i} = \mathbf{1}[(i, \text{leg}) \in \mathcal{P}^*] \cdot \left(-\frac{\ell_i}{v_i^2}\right). \quad (26)$$

For off-path legs ( $\partial C_{\max}/\partial d = 0$ ), the speed has zero effect on makespan—the AGV can be slowed arbitrarily to save energy without schedule penalty. For on-path legs, the sensitivity is  $-\ell_i/v_i^2$ : increasing speed yields diminishing returns proportional to the inverse square. Three discrete speed levels are available (Table 2): LOWER ( $v_L = 2.4$  m/s loaded), NOMINAL ( $v_N = 3.0$  m/s), and HIGHER ( $v_H = 3.6$  m/s). For on-path leg  $i$  with distance  $\ell_i$  at NOMINAL speed, the makespan reduction from upgrading to HIGHER is:

$$\Delta C_{\max} = \frac{\ell_i}{v_N} - \frac{\ell_i}{v_H} = \ell_i \left( \frac{1}{3.0} - \frac{1}{3.6} \right) = \frac{\ell_i}{18}.$$

In SD-10’s makespan-optimal schedule, task 5’s loaded leg ( $\ell = 300$  m,  $v = 3.6$  m/s [HIGHER],  $d = 83.3$  s) is the longest binding activity, contributing 18% of  $C_{\max}$ . Since it already runs at the maximum discrete speed level,  $\Delta C_{\max} = 0$  within the feasible set—no further speedup is available. TAPE identifies this saturation automatically: the highest-return intervention is not a speed change but a structural one (re-dispatching task 5 to AGV-1 to split the critical chain).

**Sensitivity to crane workload.** For QC handling activities, the duration is a fixed service time  $\tau_i$  determined by container type and crane assignment. The TAPE derivative identifies which crane services are binding:

$$\frac{\partial C_{\max}}{\partial \tau_i} = \mathbf{1}[(i, \text{qc}) \in \mathcal{P}^*]. \quad (27)$$

In  $\sigma_E^*$  (SD-10), the per-crane binding workload decomposes as:

| Crane          | On-path tasks  | Total binding time (s) | % of $C_{\max}$ |
|----------------|----------------|------------------------|-----------------|
| QC1            | tasks 0, 3     | $73 + 43 = 116$        | 12.5%           |
| QC2            | tasks 4, 7     | $45 + 30 = 75$         | 8.1%            |
| QC3            | tasks 8, 5     | $38 + 32 = 70$         | 7.6%            |
| <b>All QCs</b> | <b>6 tasks</b> | <b>261</b>             | <b>28.2%</b>    |

QC1 is the most loaded crane on the critical path, contributing 116 s (12.5% of  $C_{\max}$ ). Its two consecutive handling operations (tasks  $0 \rightarrow 3$ ) create a serialisation chain: task 3 must wait for the crane to finish task 0. Reassigning task 3 ( $\tau = 43$  s) to QC2 would remove 43 s from QC1’s binding chain if the reassignment does not create a new binding chain through QC2—a decision the terminal operator can evaluate by re-running the surrogate’s TAPE in  $< 1$  ms. This per-crane decomposition is unavailable from any global feature-importance method (e.g. TreeSHAP): it requires the *structural*, schedule-specific attribution that TAPE provides.

**Task dispatch order and critical-path formation.** The sequence in which tasks are assigned to each AGV determines the precedence chains that form the critical path. In  $\sigma_C^*$ , AGV-0’s dispatch order is  $\langle 0, 5, 4, 8, 1, 7 \rangle$ ; each consecutive pair  $(j, j')$  creates a precedence arc because  $j'$  cannot begin its empty leg until  $j$  completes delivery. TAPE reveals that *all six* consecutive pairs in this chain are binding—there is zero slack between any two successive tasks on AGV-0. Reordering the dispatch (e.g., scheduling task 5—the longest loaded leg—earlier, so it overlaps with a shorter task on AGV-1) would break this chain and potentially reduce  $C_{\max}$ .

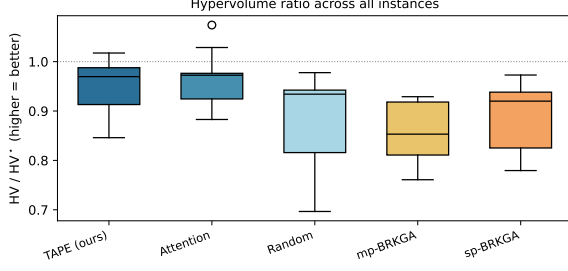
In  $\sigma_E^*$ , by contrast, the critical path threads through both AGVs and all three QCs, indicating that the dispatch order is less decisive than crane service order: even if tasks were reshuffled across AGVs, the QC serialisation would still dominate.

This constitutes the landscape analysis: a *per-solution*, *per-variable* causal attribution, at the resolution of the individual schedule rather than a global feature importance ranking, accounting for each schedule’s position on the Pareto front.

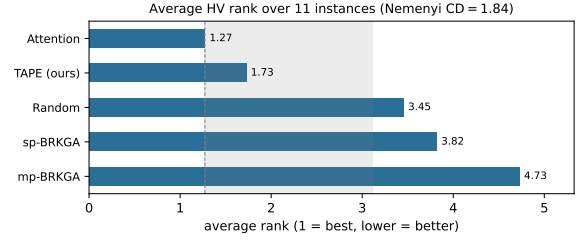
**Aggregating the landscape, and validating it against the exact oracle.** The per-schedule attribution also aggregates into the global profile a landscape analysis conventionally reports. Pooling the TAPE criticality mass of 256 sampled schedules over the four decision families named in Requirement R2—*sequence* (dispatch and QC precedence), *assignment* (which AGV), and *empty/loaded* leg speed—gives the makespan-importance profile

$$\text{assignment } 0.35 > \text{sequence } 0.26 > \text{loaded speed } 0.21 > \text{empty speed } 0.18,$$

stable to  $\pm 0.01$  across  $N \in \{10, 20, 50\}$ . The three AGV-side families (assignment and the two leg speeds) carry 0.72–0.74 of the mass against 0.26–0.28 for crane sequencing, so the instances are transport-bound—a structural property of Section 4.1 read off the attribution rather than assumed. Because the identical aggregator also runs on the exact evaluator’s critical path, the profile is checkable: at  $N=10$  and  $N=20$  the surrogate-derived and exact profiles agree in rank order and coincide numerically to  $\leq 0.0013$  per family. The cost argument is the point of doing it this way. A grouped Sobol decomposition of the same landscape needs  $\text{base} \times (2 + \text{families})$  *exact* evaluations (6,144 at  $N=10$  alone, Section 5.10) and grows with  $N$ ; the attribution route is one forward/backward pass per schedule, flat in the number of families and batchable, so the whole profile at  $N=50$ —where the exact cascade is already impractical—costs under three minutes end to end including surrogate training. One negative belongs here: the profile is nearly invariant between Pareto-optimal and dominated schedules (family-wise contrast  $\leq 0.026$ ), so this aggregate is a description of the *instance*, not a discriminator of solution quality; discriminating between front points is what the per-schedule attribution above and the trade-off scores of Section 5.11 do.



**Figure 5:** Distribution of  $HV/HV^*$  per method over all instances.



**Figure 6:** Average hypervolume rank; the shaded band is the Nemenyi critical difference.

**Extension to the coupled regime.** Under peak-power coupling, an additional class of binding variable emerges: simultaneous AGV power draw. When two AGVs attempt high-speed legs concurrently, the power budget forces one to wait, creating an additional “power arc” in the event DAG. TAPE attributes through these arcs, identifying which concurrent leg pair caused the delay and quantifying its contribution to  $C_{\max}$ . The fidelity study (Table 5) records Jaccard 0.928, 0.919 and 0.913 at  $N=10, 20, 50$ , so the landscape analysis degrades gradually under coupling. There is no closed-form critical path in this regime, so the learned surrogate is the only available source of the attribution.

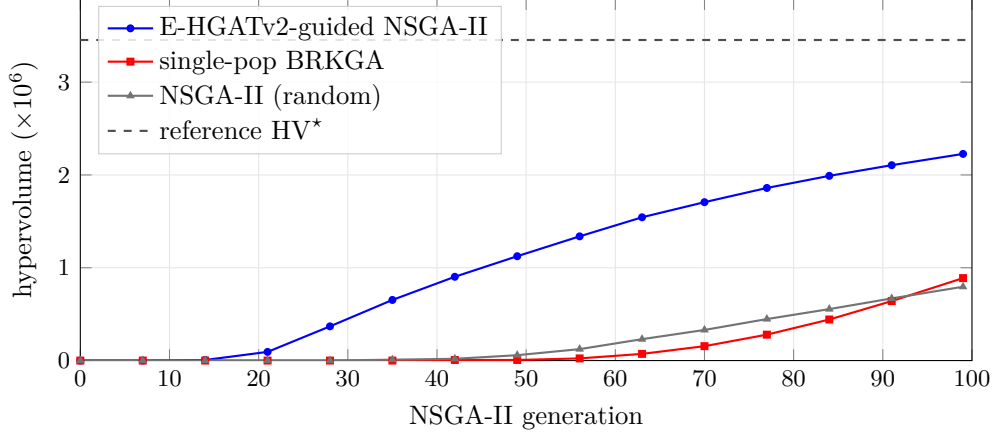
## 5.6 Optimisation quality against the metaheuristic baselines

Across the 11 instances the two graph-guided methods lead the three non-learned baselines on every indicator (Figure 5, Table 9). On the primary indicator (hypervolume ratio) the Friedman test rejects equality of the five methods with  $\chi_4^2 = 35.1$ ,  $p = 4.5 \times 10^{-7}$ ; the average ranks (Figure 6, Table 10) place the two graph-guided methods first, then the random control, then single-population BRKGA, with mp-BRKGA last. Pairwise Wilcoxon tests with Holm correction confirm that TAPE significantly beats mp-BRKGA, the random control and single-population BRKGA on hypervolume (all  $p_{\text{Holm}} < 0.004$ ), and on  $GD^+$  it beats all four including attention. In a per-instance head-to-head, TAPE achieves a higher mean hypervolume than the published mp-BRKGA on *all* 11 instances.

## 5.7 Convergence speed and scaling

The hypervolume-ratio table reports the end state of the search; the rate at which it is reached, and how the advantage depends on problem size, are reported here. Figure 7 plots the hypervolume against the NSGA-II generation on the largest instance ( $N=50$ , 100 generations, 30 seeds, all methods at the same exact-evaluation budget). The E-HGATv2-guided search reaches the hypervolume that BRKGA attains only after the full 100 generations already by generation  $\approx 42$  (a  $2.4\times$  reduction in iterations), and the unguided random control’s 100-generation value by generation  $\approx 39$ ; it then keeps climbing to a final hypervolume  $2.5\times$  that of BRKGA and  $2.8\times$  that of random. On the convergence indicator the guided search is  $\sim 5\times$  tighter to the reference front ( $IGD^+ = 296$  vs. 1435 for BRKGA and 1243 for random).

This advantage is not constant—it *amplifies with scale*. Figure 8 and Table 8 sweep  $N \in \{10, 20, 40, 80, 160\}$  in both regimes with every baseline present, at 20 seeds per cell, 40 generations, matched exact-evaluation budget,  $P = 5N$  and screening factor  $k = 4$ ; the trace above is one point of that sweep. Margins are quoted for the TAPE-guided arm, fixed in advance, the attention-guided arm overlapping it within intervals at every size. TAPE leads all four baselines at all five sizes in both regimes, and 29 of the 30 margins exceed the combined 95% confidence interval. The margin widens with  $N$ : against the unguided control  $+0.101 \rightarrow +0.584$  uncoupled and  $+0.107 \rightarrow +0.674$  coupled, against mp-BRKGA  $+0.173 \rightarrow +0.557$  and  $+0.140 \rightarrow +0.615$ .

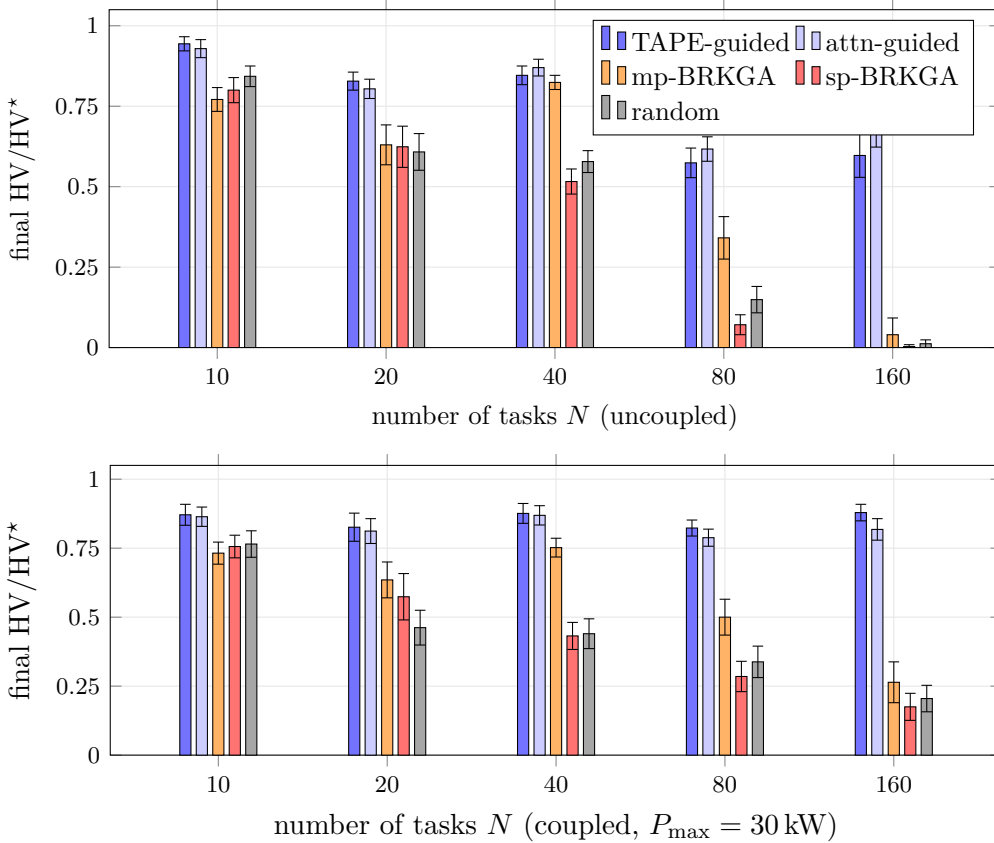


**Figure 7:** Hypervolume convergence on the largest instance of the convergence study ( $N=50$ , mean over 30 seeds, shaded bands 95% CI, dashed line the reference  $HV^*$  against which Table 8 normalises). The E-HGATv2-guided search dominates at every generation, its band separating from both baselines from generation  $\approx 18$  onwards; it matches single-population BRKGA’s *final* hypervolume by generation  $\approx 42$  and ends  $2.5\times$  higher, at  $0.64 HV^*$  against  $0.26$  and  $0.23$ . The baseline here is single-population BRKGA; mp-BRKGA enters the comparison in Table 8.

It widens because the classical baselines collapse where the guided arms do not—at  $N=160$  uncoupled, single-population BRKGA reaches  $0.003$  of  $HV^*$ , the unguided control  $0.012$  and mp-BRKGA  $0.040$ , against  $0.597$  for TAPE, so within a 40-generation budget a search without a surrogate has effectively not started at this size. At fixed budget the combinatorial volume  $|\mathcal{Z}| = A^N N! V^{2N}$  explodes and an unguided search cannot cover it, whereas the critical-path signal concentrates the same budget on the binding decisions; that concentration matters more the larger the space.

Two qualifications. Absolute quality still falls with  $N$  uncoupled ( $0.944 \rightarrow 0.597$ ), so the widening margin is relative to the baselines at matched budget, not an absolute gain; under coupling it does not fall ( $0.871 \rightarrow 0.879$ ), the power budget having contracted the reachable objective space and the reference front with it. And one margin does not separate: uncoupled  $N=40$  against mp-BRKGA ( $+0.022$ ;  $+0.123$  coupled, the two smallest in the table), a tie rather than a loss, which reverses by  $N=80$  ( $+0.233$ , TAPE ahead on 19/20 seeds against 13/20 at  $N=40$ ).

The narrowing is not a degradation of the guided arms. Between  $N=20$  and  $N=40$  the guided arms *improve* ( $0.828 \rightarrow 0.846$  TAPE,  $0.804 \rightarrow 0.870$  attention, uncoupled) while the two unguided controls decline ( $0.608 \rightarrow 0.578$  random,  $0.624 \rightarrow 0.516$  single-population BRKGA); what happens at  $N=40$  is that mp-BRKGA jumps by  $+0.194$ , more than any other method moves anywhere on the ladder, and it is that spike alone that closes the gap. Surrogate fidelity is flat across the same interval (critical-path Jaccard  $0.952$  at  $N=20$ ,  $0.978$  at  $N=40$ ,  $0.974$  at  $N=80$ ), so the model is not the cause. The likely mechanism is the instance generator’s discrete fleet provisioning: `scaled_fleet` allocates  $\max(2, \text{round}(N/12))$  AGVs and at least 3 QCs, so  $N=40$  is the single point on the ladder at which AGVs and QCs are balanced ( $AGV/QC = 1.0$ , against  $0.67$  below and  $2.33$  above) and transport contention peaks (13.3 tasks per AGV, the ladder maximum). A structure-agnostic multi-population search is relatively strongest there, and its advantage disappears once the instances become transport-bound again. We report this as the pattern the data show—a property of the instance family rather than a size limit of the surrogate—while noting that the attribution to fleet provisioning is inferred from the generator’s sizing rule and has not been isolated by a controlled re-run at fixed AGV/QC ratio.



**Figure 8:** Scaling of solution quality, 20 seeds per cell at matched exact-evaluation budget, error bars 95% CI (same runs as Table 8). Both graph-guided arms lead every baseline at every size in both regimes, and the margin widens with  $N$ : by  $N=160$  the classical baselines have effectively not converged within the 40-generation budget (uncoupled: sp-BRKGA 0.003, random 0.012, mp-BRKGA 0.040, against 0.597 for TAPE). At  $N=40$  it is mp-BRKGA that jumps rather than the guided arms that fall; see the text.

## 5.8 Optimisation cost of the faithful signal

TAPE and the attention signal are statistically indistinguishable on hypervolume: across all 11 instances at 20 seeds the Holm-adjusted TAPE-vs-attention  $p = 0.83$  (Cliff’s  $\delta = +0.01$ , negligible; TAPE ahead on 6/11), while both graph-guided variants remain well ahead of the classical baselines (TAPE > mp-BRKGA on 11/11, Holm  $p < 0.01$ ; Friedman  $p \approx 4.5 \times 10^{-7}$ ). The supported claim is therefore that routing the guidance through the faithful TAPE signal preserves the optimisation performance obtained with the attention signal, at no cost in solution quality, while the object that steers the search is also the one that explains it. The data do not support a claim that faithful guidance converges faster than attention. Section 5.2 accounts for the coincidence: both arms use the same surrogate screening, to which the mutation-targeting signal is nearly irrelevant, so the two variants are indistinguishable on optimisation. They differ in the faithfulness of the signal.

## 5.9 Statistical protocol and dispersion

We follow the Demšar [12] framework for comparing methods over multiple problem instances. Each method is run for  $S$  independent seeds per instance; the unit of analysis for the omnibus comparison is the per-instance mean. We report (i) a *Friedman* omnibus test over the five methods across the 11 instances, (ii) *average ranks* with the *Nemenyi* critical difference at  $\alpha = 0.05$ , and (iii) pairwise *Wilcoxon signed-rank* tests of TAPE against each baseline *across* instances, with *Holm–Bonferroni* correction for the four comparisons (Table 10).



**Table 8:** Optimisation ladder to  $N=160$ : final HV/HV\* after 40 generations, mean  $\pm$  95% CI over 20 seeds, matched exact-evaluation budget,  $P = 5N$ , screening factor  $k = 4$ . Best method per column in **bold**; margins are quoted for the TAPE-guided arm and  $^\dagger$  marks the one margin that does not exceed the combined interval.

| Method                     | uncoupled                 |                           |                           |                           |                           | coupled ( $P_{\max} = 30$ kW) |                           |                           |                           |                           |
|----------------------------|---------------------------|---------------------------|---------------------------|---------------------------|---------------------------|-------------------------------|---------------------------|---------------------------|---------------------------|---------------------------|
|                            | $N=10$                    | 20                        | 40                        | 80                        | 160                       | $N=10$                        | 20                        | 40                        | 80                        | 160                       |
| E-HGATv2, TAPE-guided      | <b>.944</b><br>$\pm .022$ | <b>.828</b><br>$\pm .028$ | .846<br>$\pm .029$        | .574<br>$\pm .046$        | .597<br>$\pm .068$        | <b>.871</b><br>$\pm .038$     | <b>.826</b><br>$\pm .051$ | <b>.876</b><br>$\pm .036$ | <b>.823</b><br>$\pm .029$ | <b>.879</b><br>$\pm .030$ |
| E-HGATv2, attention-guided | .929<br>$\pm .028$        | .804<br>$\pm .030$        | <b>.870</b><br>$\pm .026$ | <b>.617</b><br>$\pm .038$ | <b>.678</b><br>$\pm .055$ | .864<br>$\pm .035$            | .812<br>$\pm .045$        | .869<br>$\pm .035$        | .788<br>$\pm .031$        | .818<br>$\pm .039$        |
| mp-BRKGA                   | .771<br>$\pm .037$        | .630<br>$\pm .062$        | .824<br>$\pm .022$        | .341<br>$\pm .066$        | .040<br>$\pm .052$        | .732<br>$\pm .040$            | .635<br>$\pm .065$        | .752<br>$\pm .034$        | .500<br>$\pm .065$        | .264<br>$\pm .074$        |
| single-pop BRKGA           | .800<br>$\pm .039$        | .624<br>$\pm .064$        | .516<br>$\pm .039$        | .071<br>$\pm .031$        | .003<br>$\pm .006$        | .756<br>$\pm .041$            | .574<br>$\pm .084$        | .432<br>$\pm .049$        | .285<br>$\pm .055$        | .175<br>$\pm .049$        |
| NSGA-II (random screening) | .843<br>$\pm .032$        | .608<br>$\pm .057$        | .578<br>$\pm .034$        | .149<br>$\pm .041$        | .012<br>$\pm .012$        | .765<br>$\pm .048$            | .462<br>$\pm .063$        | .440<br>$\pm .054$        | .338<br>$\pm .057$        | .205<br>$\pm .048$        |
| TAPE – random              | +1.01                     | +2.21                     | +2.68                     | +4.25                     | +5.84                     | +1.07                         | +3.64                     | +4.35                     | +4.85                     | +6.74                     |
| TAPE – sp-BRKGA            | +1.44                     | +2.04                     | +3.30                     | +5.03                     | +5.93                     | +1.16                         | +2.52                     | +4.44                     | +5.38                     | +7.04                     |
| TAPE – mp-BRKGA            | +1.73                     | +1.98                     | +0.22 $^\dagger$          | +2.33                     | +5.57                     | +1.40                         | +1.91                     | +1.23                     | +3.23                     | +6.15                     |

**Table 9:** Main results. HV/HV\* (higher better) for all five methods, and the two faithfulness columns. Best HV per row in **bold**. Values are 20-seed means; the reference PF\* folds in every evaluated front, so HV/HV\*  $\leq 1$  by construction. The two graph-guided variants are statistically indistinguishable (TAPE is best on 6/11 rows; see Table 10), while TAPE beats every classical baseline on all 11 rows.

| Instance | Regime  | $N$ | HV/HV*       |              |        |          |          | Faithfulness |             |
|----------|---------|-----|--------------|--------------|--------|----------|----------|--------------|-------------|
|          |         |     | TAPE         | Attn         | Random | mp-BRKGA | sp-BRKGA | TAPE Jac.    | Attn $\rho$ |
| SD-5     | unc.    | 5   | <b>0.977</b> | 0.975        | 0.969  | 0.928    | 0.964    | 0.980        | 0.062       |
| SD-8     | unc.    | 8   | 0.931        | <b>0.954</b> | 0.914  | 0.881    | 0.907    | 0.990        | −0.023      |
| SD-10    | unc.    | 10  | 0.872        | <b>0.899</b> | 0.837  | 0.774    | 0.796    | 0.975        | 0.088       |
| SD-15    | unc.    | 15  | 0.866        | <b>0.869</b> | 0.784  | 0.759    | 0.806    | 0.975        | 0.081       |
| SD-20    | unc.    | 20  | <b>0.800</b> | 0.749        | 0.643  | 0.674    | 0.673    | 0.952        | −0.003      |
| L07      | unc.    | 8   | 0.951        | <b>0.959</b> | 0.928  | 0.911    | 0.921    | 0.985        | 0.176       |
| L15      | unc.    | 16  | <b>0.941</b> | 0.929        | 0.850  | 0.849    | 0.826    | 0.996        | 0.064       |
| L21      | unc.    | 12  | <b>0.940</b> | 0.916        | 0.847  | 0.742    | 0.870    | 0.988        | 0.054       |
| L35      | unc.    | 12  | <b>0.945</b> | 0.940        | 0.876  | 0.827    | 0.878    | 0.989        | 0.084       |
| SD-10-C  | coupled | 10  | 0.872        | <b>0.887</b> | 0.811  | 0.778    | 0.804    | 0.955        | −0.080      |
| SD-20-C  | coupled | 20  | <b>0.750</b> | 0.732        | 0.497  | 0.603    | 0.595    | 0.848        | −0.046      |

**Significance across instances.** The headline runs use  $S=20$  seeds on every instance, including the expensive coupled SD-20-C. The Demšar protocol aggregates the comparison over the 11 instances (where TAPE vs. mp-BRKGA reaches  $p_{\text{Holm}} = 0.0029$ ); at 20 seeds the per-instance tests are no longer floor-limited (a 5-seed two-sided Wilcoxon cannot drop below  $p = 0.0625$  by construction, which is why the original 5-seed runs leaned on the across-instance test), and the across-instance and per-instance views now agree.

**Per-instance dispersion (95% CIs).** We report per-instance uncertainty as 95% confidence intervals on HV/HV\* (Student- $t$  over the  $S$  seeds). On the well-separated instances these intervals are *non-overlapping* between the graph-guided methods and mp-BRKGA—a conservative significance criterion. For example on L21, TAPE attains  $1.017 \pm 0.028$  (CI [0.989, 1.045]) versus mp-BRKGA  $0.853 \pm 0.047$  (CI [0.806, 0.900]), with the gap far starker on the convergence indicator (GD $^+$   $4.8 \pm 1.8$  vs.  $37.8 \pm 11.1$ ); the same non-overlap holds on L15 and SD-15. On the near-optimal small instances ( $N \leq 10$ ) the intervals overlap because every method is close to the reference front.

**Table 10:** Statistical analysis over all 11 instances at 20 seeds (Friedman + average ranks; lower rank = better). Holm-adjusted pairwise  $p$ -values are TAPE vs. each method on HV/HV\*. \* denotes  $p_{\text{Holm}} < 0.05$ . TAPE vs. attention is not significant ( $p = 0.83$ )—they are statistically tied, as Section 5.2 explains, with TAPE marginally ahead on average rank.

| Method               | rank (HV) | rank (GD <sup>+</sup> ) | $p_{\text{Holm}}$ (HV) | Cliff’s $\delta$ (HV) |
|----------------------|-----------|-------------------------|------------------------|-----------------------|
| E-HGATv2-TAPE (ours) | 1.45      | 1.82                    | —                      | —                     |
| E-HGATv2-attn        | 1.55      | 1.36                    | 0.831                  | +0.01 (negligible)    |
| NSGA-II (random)     | 3.64      | 3.64                    | 0.0039*                | +0.44 (medium)        |
| single-pop BRKGA     | 3.82      | 3.18                    | 0.0020*                | +0.42 (medium)        |
| mp-BRKGA             | 4.55      | 5.00                    | 0.0029*                | +0.64 (large)         |

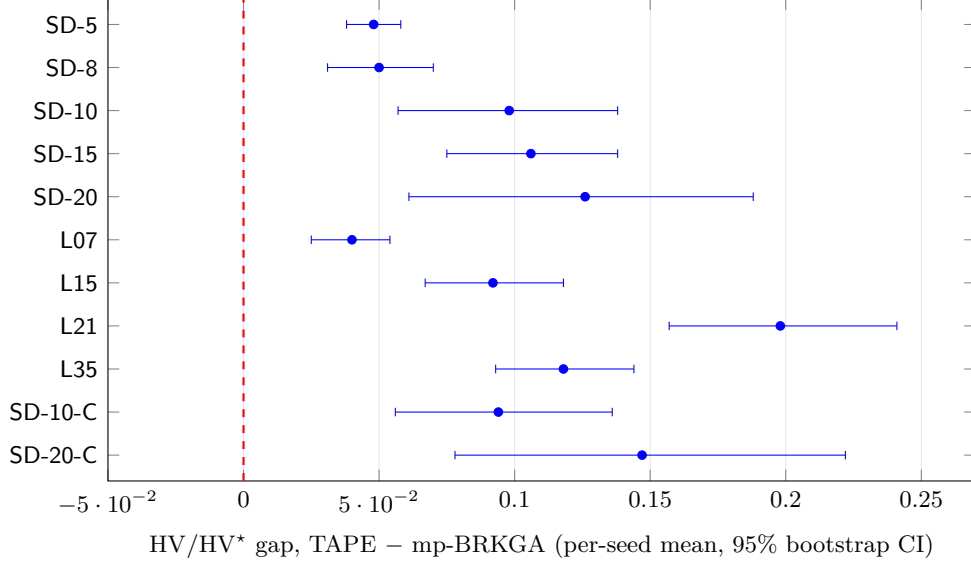
Friedman (HV):  $\chi^2_4 = 35.1$ ,  $p = 4.5 \times 10^{-7}$ ; Nemenyi  $\text{CD}_{0.05} = 1.84$ . Friedman (GD<sup>+</sup>):  $\chi^2_4 = 37.5$ ,  $p = 1.5 \times 10^{-7}$ ; Cliff’s  $\delta(\text{GD}^+, \text{TAPE vs. mp-BRKGA}) = -0.59$  (large); TAPE > mp-BRKGA on 11/11 instances.

**Table 11:** Per-instance evidence that the fused E-HGATv2 (TAPE) beats the published mp-BRKGA on HV/HV\*: the paired Wilcoxon statistic  $W$  and a percentile-bootstrap ( $10^4$  resamples) 95% CI on the per-seed gap (TAPE–mp), at 20 seeds for every instance. A CI excluding 0 is per-instance significance and holds on all 11/11 instances.

| Instance | TAPE  | mp-BRKGA | gap 95% CI (bootstrap) | $W$ | CI >0 |
|----------|-------|----------|------------------------|-----|-------|
| SD-5     | 0.977 | 0.928    | [+0.038, +0.058]       | 0   | yes   |
| SD-8     | 0.931 | 0.881    | [+0.031, +0.070]       | 12  | yes   |
| SD-10    | 0.872 | 0.774    | [+0.057, +0.138]       | 19  | yes   |
| SD-15    | 0.866 | 0.759    | [+0.075, +0.138]       | 3   | yes   |
| SD-20    | 0.800 | 0.674    | [+0.061, +0.188]       | 20  | yes   |
| L07      | 0.951 | 0.911    | [+0.025, +0.054]       | 12  | yes   |
| L15      | 0.941 | 0.849    | [+0.067, +0.118]       | 0   | yes   |
| L21      | 0.940 | 0.742    | [+0.157, +0.241]       | 0   | yes   |
| L35      | 0.945 | 0.827    | [+0.093, +0.144]       | 0   | yes   |
| SD-10-C  | 0.872 | 0.778    | [+0.056, +0.136]       | 15  | yes   |
| SD-20-C  | 0.750 | 0.603    | [+0.078, +0.222]       | 22  | yes   |

**Effect sizes.** Beyond the rank tests we quantify *how large* the advantage is with distribution-free Cliff’s- $\delta$  (Table 10, last column): against mp-BRKGA the effect is *large* ( $\delta = +0.64$  on HV and  $-0.59$  on GD<sup>+</sup>), against single-population BRKGA *medium* ( $\delta = +0.42$ ) and against the random control *medium* ( $\delta = +0.44$ ) on HV, and against attention it is *negligible* ( $\delta = +0.01$ ): TAPE matches attention while dominating every non-learned baseline by a medium-to-large margin.

**Per-instance bootstrap evidence.** We bootstrap ( $10^4$  resamples) the per-instance TAPE–mp-BRKGA hypervolume gap (Table 11, visualised as a forest plot in Figure 9). At 20 seeds the 95% CI *excludes zero on all 11 instances*—a genuine per-instance significance criterion met everywhere, including the coupled SD-20-C case (gap +0.147, CI [+0.078, +0.222]), which now clears the bar once run at the full 20-seed budget.



**Figure 9:** Per-instance bootstrap evidence ( $10^4$  resamples) for the fused E-HGATv2 (TAPE) hypervolume advantage over the published mp-BRKGA. Each point is the mean per-seed gap and its 95% CI; the dashed line marks zero. At 20 seeds the CI lies entirely to the right of zero on all 11/11 instances. This is the visual companion to Table 11.

### 5.10 Comparison with a flat tree surrogate (XGBoost + TreeSHAP)

An established alternative to a graph network is a gradient-boosted tree ensemble (XGBoost [13]) that regresses  $(C_{\max}, E)$  from the flat  $4N$  decision-variable vector, comprising for each task its sequence position, vehicle assignment and two speed levels, explained globally by TreeSHAP [14], the exact Shapley attribution for trees. This is the localised-search-guidance approach of the sequencing and speed XAI literature, and it is included here as a baseline surrogate. Three properties of the present problem determine what such a model can attribute.

**(1) Precedence structure and the  $(\max, +)$  inductive bias.** The tree ingests a fixed-length coordinate vector and does not observe the precedence DAG. The makespan is a longest path (18), and representing a max of sums over a data-dependent path with axis-aligned splits requires the ensemble to memorise regions rather than compose the recurrence. The structural signal that E-HGATv2 obtains from the fused layer ( $R^2 \approx 0.99$ , Section 5) is unavailable to it by construction, and the gap is measured rather than asserted: fitted on the same SD-10 sample the flat tree reaches held-out  $R^2 = 0.12$  on the makespan and 0.27 on energy, against 0.997 and 1.000 for the fused graph surrogate. An attribution computed on top of that fit explains a model that has not learned the objective.

**(2) The attribution target.** TreeSHAP credits decision variables: the speed level of task  $j$ , its assignment, its position. Its output is an additive credit on an input coordinate, not a derivative of the makespan with respect to a physical quantity, so the statement of Eq. 20—that delaying leg  $a$  by  $\Delta$  raises  $C_{\max}$  by exactly  $\Delta$  iff  $a \in \pi^*$ —has no counterpart in it, and  $\partial C_{\max} / \partial(\text{leg})$  is not defined. The two methods therefore attribute to different objects: decision variables in one case, physical activities in the other.

**(3) The landscape is interaction-dominated.** A Sobol variance decomposition [18] on SD-10 (1024 base samples, 6,144 evaluations) gives the sequence and assignment variables first-order indices of 0.00 for the makespan against total-order indices of 0.71 and 0.72 respectively, so their influence is carried almost entirely through interactions, with negligible main effects (for energy,

**Table 12:** A flat tree surrogate with TreeSHAP and the physics-fused graph surrogate with TAPE, compared on the requirements of this problem.

| Capability                                                        | XGBoost + TreeSHAP         | E-HGATv2 + TAPE         |
|-------------------------------------------------------------------|----------------------------|-------------------------|
| Input respects precedence DAG                                     | no (flat $4N$ vector)      | yes (graph)             |
| Makespan inductive bias                                           | none (axis splits)         | (max, +) longest path   |
| Attribution target                                                | decision variable          | physical activity / leg |
| Critical-path derivative $\partial C_{\max}/\partial(\text{leg})$ | not defined                | exact (Eq. 20)          |
| Faithfulness vs. exact critical path                              | n/a (no path)              | Jaccard 0.95–1.00       |
| Interaction-dominated effects                                     | additive credit (mismatch) | joint critical path     |
| Routes operator type (AGV vs. QC)                                 | no                         | yes (Sec. 3.8)          |

total-order 0.75/0.57 against first-order 0.36/0.31). TreeSHAP’s per-feature mean-|SHAP| is an additive, main-effect-like decomposition, and where main effects vanish and interactions dominate that credit is divided among the interacting variables without localising the bottleneck. The divergence is measurable on the same instance: Sobol places 0.94 of the makespan importance mass on the two structural families (sequence, assignment), whereas TreeSHAP places only 0.51 there and spills the remainder onto the speed coordinates—an absolute under-weighting of 0.43, and the same 0.43 for energy. TAPE attributes instead to the jointly binding activity, the critical path, which is the interaction the Sobol indices identify.

A tree that fits the scalar objective accurately still attributes to input coordinates rather than to activities: it identifies no binding leg, supplies no makespan derivative, and divides its additive importances among interacting variables in the regime the Sobol indices describe (Table 12). The graph surrogate supplies an explanation with the three properties the analysis requires: consistency with the physics, localisation on an activity, and a form the search can consume.

### 5.11 Behaviour of the Pareto front

The same object is now applied to the front rather than to a single schedule. For each instance we take the approximated Pareto set produced by the guided search, read the local trade-off weight  $\lambda$  at every front point, and compute the Trade-off Criticality Score of Section 3.9. Table 13 and Figure 10 report, per instance: the range of  $\lambda$  actually swept along the front; the top bottleneck tasks at the makespan-optimal extreme versus the energy-optimal extreme; the *bottleneck migration* (1 minus the Jaccard overlap of the top-3 tasks at the two extremes, so larger means the binding bottleneck genuinely changes as one walks the front); and the *TCS concentration* (the share of the total trade-off criticality carried by the top-3 tasks). Two patterns emerge. First, the criticality is concentrated on a few tasks—the top three carry 27–87% of the total criticality (most concentrated at small  $N$ )—so each front point has a short, human-readable explanation. Second, on 9 of the 11 instances the binding bottleneck *migrates* between the makespan-optimal and energy-optimal ends of the front (migration  $\geq 0.5$ ): this is exactly the “why is this solution Pareto-optimal” knowledge different solutions are non-dominated because different operations bind their objectives. The two exceptions (SD-10, SD-20-C) keep the same three top tasks across the front. The  $\lambda$  range column confirms the runs actually span the trade-off, from near-energy-optimal ( $\lambda \approx 0$ ) to makespan-optimal ( $\lambda = 1$ ).

### 5.12 Transferring front behaviour across instances

Section 5.11 describes each front with a faithful object. A stronger property is transfer: that a single model, given only an instance’s structure, predicts its front composition without running the search. This is tested with one lightweight predictor  $g : (\text{structural features}, \lambda) \mapsto \rho$ , where

**Table 13:** Pareto-front behaviour via TAPE Trade-off Criticality Scores. “front” = number of non-dominated points; “ $\lambda$  range” = trade-off weight swept along the front; “top-3” = the three highest-TCS (most binding) tasks at the makespan-optimal ( $C_{\max}$ ) and energy-optimal ( $E$ ) extremes; “migr.” =  $1 - \text{Jaccard}$  of those two task sets (higher = the bottleneck changes more along the front); “conc.” = share of total criticality on the top-3 tasks.

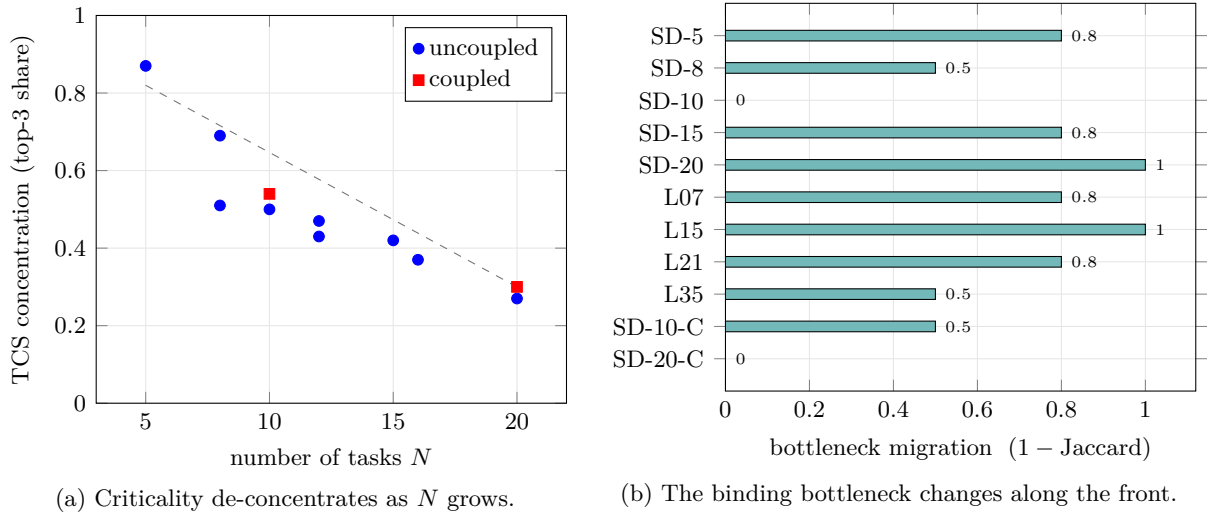
| Instance | $N$ | front | $\lambda$ range | top-3 ( $C_{\max}$ end) | top-3 ( $E$ end) | migr. / conc. |
|----------|-----|-------|-----------------|-------------------------|------------------|---------------|
| SD-5     | 5   | 82    | [0.23, 1.00]    | {2, 3, 1}               | {0, 2, 4}        | 0.80 / 0.87   |
| SD-8     | 8   | 107   | [0.30, 1.00]    | {2, 3, 5}               | {2, 3, 4}        | 0.50 / 0.69   |
| SD-10    | 10  | 115   | [0.25, 1.00]    | {0, 1, 2}               | {0, 1, 2}        | 0.00 / 0.50   |
| SD-15    | 15  | 28    | [0.18, 1.00]    | {2, 4, 5}               | {0, 1, 4}        | 0.80 / 0.42   |
| SD-20    | 20  | 57    | [0.03, 1.00]    | {0, 1, 3}               | {2, 4, 6}        | 1.00 / 0.27   |
| L07      | 8   | 55    | [0.14, 0.98]    | {2, 4, 6}               | {0, 1, 2}        | 0.80 / 0.51   |
| L15      | 16  | 44    | [0.25, 1.00]    | {3, 6, 10}              | {0, 1, 5}        | 1.00 / 0.37   |
| L21      | 12  | 39    | [0.49, 0.99]    | {0, 3, 10}              | {0, 2, 4}        | 0.80 / 0.47   |
| L35      | 12  | 54    | [0.25, 1.00]    | {0, 1, 4}               | {0, 4, 8}        | 0.50 / 0.43   |
| SD-10-C  | 10  | 88    | [0.38, 1.00]    | {0, 1, 2}               | {0, 1, 4}        | 0.50 / 0.54   |
| SD-20-C  | 20  | 44    | [0.00, 1.00]    | {0, 1, 2}               | {0, 1, 2}        | 0.00 / 0.30   |

$\rho$  is the transport share of the critical path (the composition of Table 7) and the features are  $N$ , the AGV and QC counts, and the mean QC handling time. Evaluation is by leave-one-instance-out (LOIO): train on all but one instance, predict the held-out instance’s front, and compare against a constant baseline that ignores all features and emits the training-set mean  $\bar{\rho}$ . Improvement over that baseline is the criterion for structure having been used.

**Discriminative power of the public benchmark.** On the 35 instances L01–L35 (the small set of [3]), LOIO attains a held-out  $\text{MAE}(\rho) = 0.048$ . The constant baseline attains the same 0.048, and the learned model improves on it for 14/35 instances, so the features carry no usable signal on this set. The target is saturated: across all 35 instances and every front point the critical path is transport-bound ( $\rho = 0.88 \pm 0.04$  between instances, with 87% of all front points at  $\rho > 0.80$ ). With no composition variance to predict, no predictor can improve on a constant, and these instances therefore cannot test transfer in either direction. The saturation is itself informative: for this terminal layout, transport binding the critical path is a near-invariant structural property.

**Transfer under varying front composition.** To obtain instances whose binding resource differs, the fleet-to-crane ratio is varied: many AGVs with few cranes makes transport cheap and QC handling bind ( $\rho$  small), while few AGVs with many cranes makes transport bind ( $\rho \rightarrow 1$ ). A 20-instance set built this way spans  $\rho \in [0.19, 0.99]$  (between-instance std 0.29, versus 0.04 on the public set). On it, LOIO separates the two (Table 14):  $\text{MAE}(\rho) = 0.107$  against a constant baseline of 0.289, a factor of 2.7, with the model ahead on 17/20 instances, and the predicted and true per-instance transport shares correlate at  $r = 0.945$ . A single surrogate, given only an unseen instance’s fleet and crane structure, places it on the QC-bound ( $\rho = 0.19$ ) to transport-bound ( $\rho = 0.99$ ) axis.

**Targets that do not transfer.** Two finer targets resist prediction, and we report them. (i) The *within-front* ordering of  $\rho$  along a single instance’s front is recovered only weakly (median per-instance correlation  $\approx 0.12$ ): real fronts are compositionally near-flat—the within-front standard deviation of the transport share is 0.048 on average and below 0.06 for 80% of instances, against a between-instance spread of 0.29—so there is little ordering signal to recover. (ii) Predicting which task is on the critical path from static task features is at chance—LOIO



**Figure 10:** Pareto-front behaviour from TAPE Trade-off Criticality Scores. (a) the share of total criticality borne by the three most-binding tasks falls from  $\approx 0.87$  at  $N=5$  to  $\approx 0.30$  at  $N=20$ —larger fronts spread the binding constraints over more operations. (b) on 9 of 11 instances the top-3 binding tasks at the makespan-optimal end of the front differ substantially from those at the energy-optimal end (migration  $\geq 0.5$ ): *different operations make different front points Pareto-optimal*.

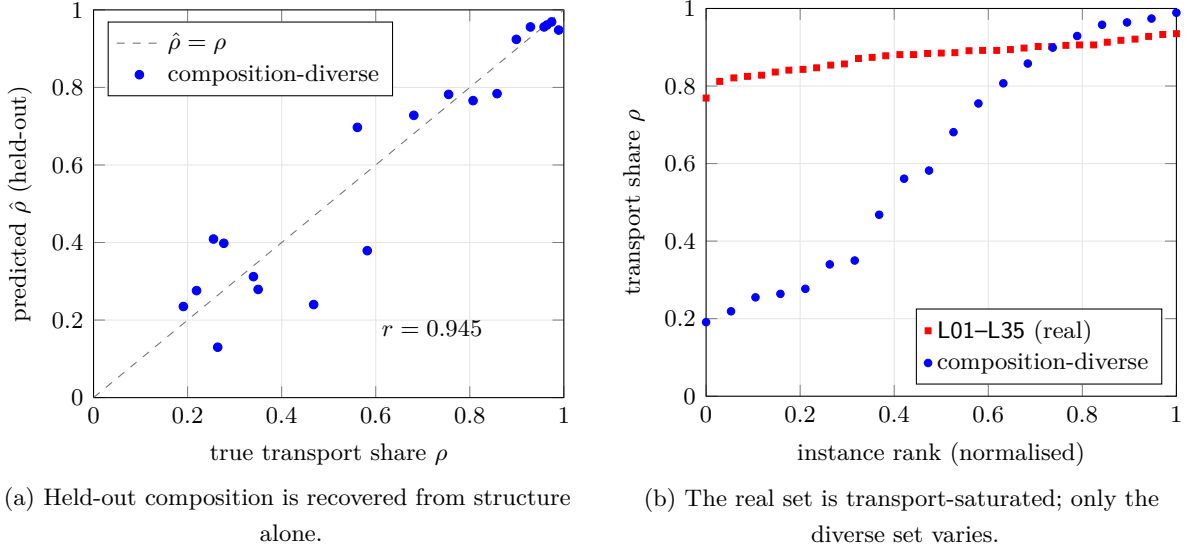
**Table 14:** Transfer of front composition, leave-one-instance-out. The model beats a constant-mean baseline only where the target  $\rho$  (transport share of the critical path) varies. The public small set is transport-saturated and does not discriminate between the two; on a composition-spanning set the structural predictor recovers  $\rho$  across unseen instances at  $r = 0.945$ .

| Instance set                    | #inst | $\rho$ spread<br>(std / range) | LOIO MAE( $\rho$ ) |          | between-inst.<br>recovery $r$ |
|---------------------------------|-------|--------------------------------|--------------------|----------|-------------------------------|
|                                 |       |                                | model              | constant |                               |
| L01–L35 (real, small)           | 35    | 0.04 / [0.77, 0.94]            | 0.048              | 0.048    | — (saturated)                 |
| composition-diverse (synthetic) | 20    | 0.29 / [0.19, 0.99]            | <b>0.107</b>       | 0.289    | <b>0.945</b>                  |

AUC = 0.53, no better than a  $\lambda$ -only baseline—because task-level criticality is set by the realised schedule, not by static attributes. The amortisation benefit is therefore at the level of *instance composition*, not per-task criticality. (iii) The transfer is interpolative in fleet structure. Two grouped holdouts test this directly. Holding out an entire crane count—train on 16 instances with 2–6 quay cranes, test on the unseen 5-crane instances plus the real L07—gives test MAE( $\rho$ ) = 0.113 at  $r = 0.44$ , against 0.049 / 0.87 in training. Removing the small-fleet end as well, so training spans only 3–6 cranes and the test set contains 2-crane instances below that range, degrades the test correlation to zero (MAE = 0.158,  $r = -0.01$ ), and the damage is concentrated exactly on the out-of-range instances (toy:24:2:2 0.242, toy:40:2:3 0.237, versus 0.113–0.161 for the in-range ones). The predictor places an unseen instance on the composition axis when its fleet structure is bracketed by the training set and not when it falls outside; scaling to genuinely novel terminal configurations needs training coverage, not just more instances. Finally, the composition-spanning evidence is synthetic; confirming it on a real, structurally diverse benchmark (e.g. the large DL set of [3], which varies AGV and QC counts) is the remaining step.

### 5.13 Fidelity and faithfulness under peak-power coupling

Under the peak-power budget all methods degrade, and the faithful signal is slightly behind attention (Table 9, SD-10-C/SD-20-C). The surrogate’s held-out  $R^2$  falls to  $\approx 0.80$  (from 0.99 uncoupled) and TAPE’s *front-averaged* leg-critical Jaccard drops to 0.87 at  $N = 20$ . Both



**Figure 11:** Transfer of front composition. **(a)** On a composition-spanning set, leave-one-instance-out predictions of the transport share  $\rho$  track the truth ( $r = 0.945$ ): a single surrogate reads an unseen instance’s fleet/crane structure and places it on the QC-bound/transport-bound axis. **(b)** Why the public benchmark cannot show this: across all 35 real instances  $\rho = 0.88 \pm 0.04$  (near-flat, transport-bound), so a constant matches any predictor; the synthetic set deliberately spans  $\rho \in [0.19, 0.99]$ .

graph-guided methods still beat all three non-learned baselines, but the coupled claim is the weakest.

**Behaviour at the makespan-optimal extreme.** The front-averaged number hides a sharp, mechanistically understood failure at *one* end of the front. On SD-10-C, the energy-optimal extreme is recovered exactly (leg-Jaccard 1.00), but the makespan-optimal extreme collapses to 0.11. This is where every AGV runs at maximum speed, so power contention is maximal and the binding critical path is set by the disjunctive power-wait arcs that have no closed form—the structure the fused head must learn, and the regime where its makespan  $R^2$  is lowest (0.90 on this instance). A rerun with a deeper physics unroll (4 steps) and 2000 samples reproduced the *identical* 0.11, locating the cause in the surrogate’s accuracy ceiling at maximum contention rather than in the sample size. We therefore report the front-averaged Jaccard as the headline and record the makespan-extreme degradation as a limitation: coupled attribution is faithful across most of the front and not at the maximum-power-contention corner, which a richer power-wait head or contention-aware sampling would address.

## 6 Discussion and limitations

We delimit what the evidence does and does not support.

- **Reference front.** Hypervolume is reported against a high-budget PF<sup>\*</sup> formed as the non-dominated union of long reference runs and every evaluated front, so  $HV/HV^* \leq 1$  by construction (verified: no cell exceeds 1.0). The true front is unknown at these sizes, as is standard in EMO practice.
- **Seeds and budget.** The headline runs use 20 seeds on every instance, including the coupled SD-20-C; at that resolution the TAPE-vs-attention hypervolume difference is a genuine statistical tie ( $p_{\text{Holm}} = 0.83$ ) rather than an underpowered one, because attention’s search benefit is the shared surrogate screening and not the attention map itself (Section 5.2). The

per-population size is  $5N$  rather than the reference  $20N$  for tractability, with all methods held to the same per-generation exact-evaluation budget; scaling either up is straightforward.

- **Generalisation.** A single surrogate does transfer front composition to unseen instances when composition varies—leave-one-instance-out recovery  $r = 0.945$  on a composition-spanning set (Section 5.12, Figure 11). Two qualifications apply: the public small benchmark is transport-saturated and therefore cannot exhibit the effect (a constant matches any predictor there), and the composition-spanning demonstration is synthetic. Confirming amortisation on a real, structurally diverse set (the large DL instances of [3]) is the natural next step; per-task criticality, by contrast, is set by the realised schedule and does not amortise.

## 7 Conclusion

A single faithful object—the gradient of an exact max-plus longest path embedded in a heterogeneous graph surrogate—serves simultaneously as an explanation (R1, R2), a search controller (R3), and a description of Pareto-front behaviour (R4) for bi-objective quay-crane and SA-AGV scheduling. It is provably faithful where learned attention is not, and routing the search through it preserves a statistically significant advantage over the published mp-BRKGa baseline. The remaining work—confirming amortisation on a real, structurally diverse instance set, strengthening the coupled surrogate at the makespan extreme, and reproducing the published optima on the DS/DL benchmark—is incremental and well scoped.

## A Implementation of the tropical layer

The  $(\max, +)$  longest-path layer of Section 3.5 is implemented as a hand-written `torch.autograd.Function` rather than as a composition of generic autograd primitives. The forward pass relaxes nodes in topological order and caches the argmax incoming arc of every node; the backward pass routes the incoming cotangent along that cached path, assigning unit sensitivity to the arcs and nodes on the binding path and exactly zero elsewhere. This realises (21) directly, at the cost of one traversal, and renders the attribution binary rather than a softened approximation.

A batched variant evaluates a block-diagonal stack of schedule DAGs in a single vectorised relaxation. Because the blocks share no arcs,  $\nabla \sum_i C_{\max}^{(i)}$  decomposes exactly into the per-schedule subgradients with no cross terms, so a whole generation’s offspring are attributed in one pass; the batched and per-graph paths are asserted bit-identical by a parity test in both regimes.

## B Hyper-parameters, model size and environment

**Environment.** Python 3.12.11, PyTorch 2.6.0, PyTorch Geometric 2.8.0. All reported results were produced on a rented Linux host with 255 CPU cores, 2 TB of RAM and two NVIDIA A40 (46 GB) accelerators. The surrogate is nonetheless executed on CPU throughout: the scatter-max aggregation and the  $(\max, +)$  relaxation are order-sensitive under non-deterministic GPU reductions, and CPU execution makes every number bit-reproducible from its recorded seed. The GPUs were used only to profile the batched tropical dynamic program and are not required to reproduce any reported number. Throughput comes instead from process-level parallelism: search experiments were sharded across cores, one single-threaded process per (instance, seed-block) pair. The source code, the instance data, the seeds and the result artifacts from which every reported table and figure is regenerated are available at <https://github.com/aayushjha1729/e-hgatv2>.



**Table 15:** Architecture and optimisation settings. The heterogeneous core is trained on exact evaluator labels; the fused physics heads are then fitted with the core frozen.

| Component           | Setting                                         | Value                                           |
|---------------------|-------------------------------------------------|-------------------------------------------------|
| E-HGATv2 core       | message-passing layers                          | 3                                               |
|                     | hidden width $d$                                | 64                                              |
|                     | attention heads (width $d/4$ each)              | 4                                               |
|                     | relation types (AGV, QC)                        | 2                                               |
|                     | trainable parameters                            | 73,410                                          |
| Core optimisation   | optimiser                                       | Adam                                            |
|                     | learning rate                                   | $1 \times 10^{-3}$                              |
|                     | weight decay                                    | $1 \times 10^{-5}$                              |
|                     | epochs                                          | 50                                              |
|                     | batch size (graphs)                             | 32                                              |
| Fused physics heads | trainable parameters (uncoupled)                | 12,867                                          |
|                     | trainable parameters (coupled, incl. wait head) | 22,213                                          |
|                     | core parameters during fine-tune                | frozen                                          |
|                     | optimiser / schedule                            | Adam, cosine annealing                          |
|                     | learning rate ( $\rightarrow \eta_{\min}$ )     | $1 \times 10^{-3} \rightarrow 1 \times 10^{-5}$ |
|                     | epochs                                          | 30                                              |
|                     | self-supervised training schedules per instance | 800                                             |

**Cost.** The tropical layer adds  $\Theta(N)$  arcs and therefore  $\mathcal{O}(N)$  work per evaluation (approximately  $4N$  multiply-adds at the reported sizes), which is negligible beside message passing. Wall-clock timings for search and screening are reported in Section 5.2.

**Search configuration of the headline runs.** Table 16 records the settings behind Table 10 and Section 3.8. Every instance is run at the same generation count, seed count and screening factor; the population differs across instances only through  $N$ .

## C The synthetic dual-cycling instance family

The results rest on two instance families of different provenance. The L-instances are the published benchmark: both the QC $\leftrightarrow$ LU distance matrix and the per-crane task lists are read from Tables 4 and 5 of [3] without modification. The SD family is generated here; its inherited and constructed components are separated below.

**Inherited components.** The vehicle physics is not synthetic. Both the kinematics  $v = \alpha v_0$  and the per-leg power draws are the published constants of Table 1, taken from [1, 2]; the implementation asserts the kinematic relation at import time and fails if a constant drifts. Travel times are  $t = d/v$  and leg energies  $E = \varepsilon t$  over the same published asymmetric distance matrix used by the L-instances, so an SD instance moves containers over real terminal geometry (nodes QC1–QC6, LU1–LU6) at real speeds and powers. The timing recurrence and the objectives are likewise those of Section 2.3, unchanged.

**Constructed components.** Only the task list is synthetic, and it is generated by a deterministic rule rather than sampled from a demand model. For  $N$  tasks indexed  $j = 0, \dots, N - 1$

**Table 16:** Search configuration of the 11 headline instances.  $P$  is the mp-BRKGA per-population size and  $4P = (\Omega + \Pi)P$  the matched population of every other method, so all five methods spend the same number of exact evaluations per generation. Seeds are executed as five shards of four seeds each and merged.

| Instance                                      | $N$ | Regime          | $P = 5N$        | Matched pop. $4P$ | Ref. HV runs |
|-----------------------------------------------|-----|-----------------|-----------------|-------------------|--------------|
| SD-5                                          | 5   | uncoupled       | 25              | 100               | 50 gen.      |
| SD-8                                          | 8   | uncoupled       | 40              | 160               | 50 gen.      |
| SD-10                                         | 10  | uncoupled       | 50              | 200               | 50 gen.      |
| SD-15                                         | 15  | uncoupled       | 75              | 300               | 50 gen.      |
| SD-20                                         | 20  | uncoupled       | 100             | 400               | 50 gen.      |
| SD-10-C                                       | 10  | coupled (30 kW) | 50              | 200               | 50 gen.      |
| SD-20-C                                       | 20  | coupled (30 kW) | 100             | 400               | 50 gen.      |
| L07                                           | 8   | uncoupled       | 40              | 160               | 50 gen.      |
| L21                                           | 12  | uncoupled       | 60              | 240               | 50 gen.      |
| L35                                           | 12  | uncoupled       | 60              | 240               | 50 gen.      |
| L15                                           | 16  | uncoupled       | 80              | 320               | 50 gen.      |
| <i>Common to all instances:</i>               |     |                 |                 |                   |              |
| generations $G$                               |     |                 | 40              |                   |              |
| seeds $S$ (5 shards $\times$ 4)               |     |                 | 20              |                   |              |
| screening factor $\kappa$                     |     |                 | 2               |                   |              |
| mutation temperature $T$                      |     |                 | 0.25            |                   |              |
| crossover / mutation probability              |     |                 | 0.9 / 0.9       |                   |              |
| crossover inheritance bias $p_e$              |     |                 | 0.7             |                   |              |
| tournament size                               |     |                 | 2               |                   |              |
| mp-BRKGA elite / mutant fraction              |     |                 | 0.2 / 0.1       |                   |              |
| mp-BRKGA $\Omega$ / $\Pi$ / exchange interval |     |                 | 2 / 2 / 30 gen. |                   |              |
| faithfulness sample per instance              |     |                 | 40 schedules    |                   |              |

with crane set  $\mathcal{Q}$  and stations LU1, ..., LU6:

$$\begin{aligned}
 \text{crane}(j) &= \mathcal{Q}[j \bmod |\mathcal{Q}|], & \text{station}(j) &= \text{LU}[(j \bmod 6) + 1], \\
 \text{kind}(j) &= \begin{cases} \text{LOAD} & j \text{ even} \\ \text{UNLOAD} & j \text{ odd} \end{cases}, & \tau_j &\sim \mathcal{U}\{30, \dots, 80\} \text{ s},
 \end{aligned}$$

with all vehicles initially parked at LU1 and  $\tau_j$  drawn from a seeded `numpy` generator, so the instance is a pure function of its seed. Fleet size is a pure function of  $N$  as well,  $|\mathcal{A}| = \max(2, \text{round}(N/12))$  and  $|\mathcal{Q}| = \text{clip}(\text{round}(N/40), 3, 6)$ ; determinism here is a requirement rather than a convenience, because the search process and the reference-front process construct the instance independently and must obtain the identical object.

**Purpose and limitations of the family.** It supplies three properties the published set does not. First, dual cycling: every move in the L-instances is a **LOAD**, so **LOAD/UNLOAD** alternation, the setting treated in this paper, is not exercised by them at all. Second, an exactly enumerable reference: at  $N = 5$  the Pareto front is computed by brute force, giving a  $\text{PF}^*$  that is the true front rather than a high-budget approximation. Third, a size ladder to  $N = 160$  (Section 5.7) at fixed structure, which the four published instances ( $N = 8\text{--}16$ ) cannot provide.

Two properties distinguish the family from the published task lists. *(i) Station spread.* Each L-instance draws on one to three distinct LU stations, with LU1–LU3 carrying 61% of all 368 published moves; the round-robin rule here cycles all six. Travel therefore samples more of the distance matrix than the published instances do, which raises the travel share of the critical path relative to handling. *(ii) Dual cycling.* The alternation itself is unvalidated, every published move being a **LOAD**. Crane loading is not a point of difference: the published instances assign

an equal number of moves to every crane in all 35 cases, and the round-robin rule reproduces that, with per-crane *handling-time* imbalance of a comparable magnitude in both (0.21 mean relative spread on the published set, 0.31 here). Every claim that requires realistic *structure*—rather than realistic physics—is therefore reported on the L-instances, and obtaining the DS/DL dual-cycling task lists of [3] is the item that would let the dual-cycling results be restated on published data.

**Coupled variants.** The SD-*N*-C instances add the peak-power budget of Section 2.4 to an otherwise identical SD-*N* instance. The peak-power model follows [4]; no published instance set states it over AGV travel legs in the terminal setting, so these instances supply the coupled evaluation. The budget  $P_{\max} = 30$  kW is chosen above the largest single-leg draw (19.8 kW), so every leg remains individually feasible and infeasibility can only arise from concurrency.

## D Software architecture

This appendix documents the implementation, since several claims in the paper concern what the code computes and are checkable only against it. The source tree described here is the one released with the paper (Appendix B).

**Stack.** The system is a Python package (`ehgat`,  $\approx 9.8$ k lines across the eight modules of Table 17, with  $\approx 3.4$ k further lines of tests). Message passing uses PyTorch 2.6 with PyTorch Geometric 2.8; the environment, evaluator, Pareto enumeration and BRKGA baselines depend only on `numpy` and `scipy` and import no learning framework, which is what allows the exact evaluator to be tested independently of the surrogate. Statistics use `scipy.stats` (Wilcoxon, Friedman); the tree-surrogate comparison of Section 5.10 uses `xgboost` and `shap`.

**Table 17:** Package structure. The dependency direction is downward: no module imports a module above it, so the exact physics layer is independent of the learned layers.

| Module                   | Torch | Responsibility                                                                                                                                             |
|--------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>environment</code> | no    | Instance construction, distance matrix, published speed/power constants, random-key decoder, exact evaluator (both regimes), exhaustive Pareto enumeration |
| <code>baselines</code>   | no    | Single-population BRKGA and mp-BRKGA                                                                                                                       |
| <code>metrics</code>     | no    | Hypervolume, $GD^+$ / $IGD^+$ , spread                                                                                                                     |
| <code>surrogate</code>   | yes   | Schedule $\rightarrow$ <code>HeteroData</code> encoding, E-HGATv2 core, training loop, attention-control models                                            |
| <code>explain</code>     | yes   | Event DAG assembly, tropical DP layers, fused model, TAPE extraction, Trade-off Criticality Scores                                                         |
| <code>search</code>      | yes   | NSGA-II, the guided/screened variant, TAPE guidance signals                                                                                                |
| <code>benchmark</code>   | yes   | Experiment drivers, faithfulness scoring, statistics, landscape studies, plots                                                                             |
| <code>utils</code>       | no    | Seeding, semantic tensor assertions                                                                                                                        |

**The tropical layer as an autograd primitive.** The  $(\max, +)$  recurrence is implemented as a subclass of `torch.autograd.Function` whose `forward` relaxes nodes in topological order and stores the argmax in-arc of each node, and whose `backward` walks that stored structure to route the cotangent (Appendix A), in place of a composition of framework operators. Two

consequences matter for the paper’s claims. Since the  $\text{argmax}$  is stored rather than recomputed from a differentiable surrogate of the maximum, the returned attribution is exactly the binary indicator of (20) and not a softened approximation; and since the backward pass is a single traversal of the stored path, the attribution costs one  $O(N)$  pass rather than the  $O(4N)$  perturbation runs a black-box evaluator would need. A second implementation evaluates a block-diagonal stack of schedule DAGs in one vectorised relaxation, and a parity test asserts that the two agree bit-for-bit in both regimes.

**Where the surrogate enters the optimiser.** Screening is implemented as an over-produce-and-rank step inside the generation loop:  $\kappa\mu$  offspring are generated, the surrogate predicts  $(C_{\max}, E)$  for all of them in one batched forward pass, they are ordered by non-dominated rank and crowding distance under the *predicted* objectives, and only the best  $\mu$  are passed to the exact evaluator. The exact-evaluation counter is incremented by  $\mu$ , which is what makes the matched-budget claim of Section 4 an identity in the code rather than an approximate accounting. Screening consumes the fused head’s regression output only; the attribution signal is used solely for mutation targeting, so the two mechanisms are separately switchable and are separately ablated in Section 5.2. The same screening step is exposed as an injectable argument of the mp-BRKGA implementation, which allows the surrogate to be inserted into the published algorithm rather than only compared against it.

**Semantic tensor assertions.** Feature vectors carry a fixed, named order (nodes: handling time, LOAD indicator, UNLOAD indicator, crane index; arcs: travel time, empty energy, loaded energy). Every tensor crossing a module boundary is checked against that named signature, so a transposed or reordered feature raises rather than degrading accuracy—a failure mode that is otherwise invisible in a regression metric.

**Determinism.** A single integer seeds Python, `numpy` and `PyTorch`, and each stochastic component draws from its own derived generator so that, for example, instance generation and BRKGA evolution cannot perturb one another. `PyTorch` runs with deterministic algorithms enabled and on CPU throughout: the scatter-max aggregation and the  $(\max, +)$  relaxation are order-sensitive, and non-deterministic GPU reductions would break exact reproducibility of both the makespan and the  $\text{argmax}$  path. Search experiments are parallelised by sharding over (instance, seed-block) pairs, one process per shard, which changes only the assignment of seeds to processes and not the per-seed result.

**Verification.** The test suite is 239 tests over 29 modules and is the basis for the exactness claims. The evaluator is checked against a literal line-by-line transcription of the MILP timing constraints solved by fixed-point longest-path iteration, and against exhaustive enumeration of all  $3^{2N}$  speed assignments at small  $N$ ; the decoder is checked to be a left inverse of the encoder; the differentiable DP is checked against the simulator’s makespan; the batched tropical layer is checked against the per-graph one; and the coupled simulator is checked to reduce to the uncoupled evaluator when the power budget never binds.

## References

- [1] D. B. M. M. Fontes and S. Homayouni. *A bi-objective multi-population biased random key genetic algorithm for joint scheduling of quay cranes and speed-adjustable vehicles in container terminals*. Flexible Services and Manufacturing Journal, 35:241–268, 2022.
- [2] D. B. M. M. Fontes and S. Homayouni. *Joint scheduling of quay cranes and speed-adjustable automated guided vehicles in container terminals*. (Single-indexed MILP timing model), 2023.

- [3] S. Homayouni and D. B. M. M. Fontes. *Energy-efficient scheduling of quay cranes and automated guided vehicles* (loading benchmark, Tables 4–5), book chapter, 2022.
- [4] S. Homayouni and D. B. M. M. Fontes. *Optimizing job shop scheduling with speed-adjustable machines and peak power constraints: a mathematical model and heuristic solutions*. International Transactions in Operational Research, 32(1):194–220, 2025.
- [5] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. *A fast and elitist multiobjective genetic algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation, 6(2):182–197, 2002.
- [6] H. Ishibuchi, H. Masuda, Y. Tanigaki, and Y. Nojima. *Modified distance calculation in generational distance and inverted generational distance*. EMO 2015, LNCS 9019:110–125.
- [7] S. Brody, U. Alon, and E. Yahav. *How attentive are graph attention networks?* ICLR 2022.
- [8] P. Veličković et al. *Graph attention networks*. ICLR 2018.
- [9] X. Wang et al. *Heterogeneous graph attention network*. WWW 2019.
- [10] S. Jain and B. C. Wallace. *Attention is not explanation*. NAACL 2019.
- [11] F. Baccelli, G. Cohen, G. J. Olsder, and J.-P. Quadrat. *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Wiley, 1992.
- [12] J. Demšar. *Statistical comparisons of classifiers over multiple data sets*. Journal of Machine Learning Research, 7:1–30, 2006.
- [13] T. Chen and C. Guestrin. *XGBoost: A scalable tree boosting system*. KDD 2016, 785–794.
- [14] S. M. Lundberg et al. *From local explanations to global understanding with explainable AI for trees*. Nature Machine Intelligence, 2:56–67, 2020.
- [15] J. E. Kelley and M. R. Walker. *Critical-path planning and scheduling*. Eastern Joint Computer Conference, 1959, 160–173.
- [16] A. Mensch and M. Blondel. *Differentiable dynamic programming for structured prediction and attention*. ICML 2018, 3462–3471.
- [17] A. M. Rush. *Torch-Struct: Deep structured prediction library*. ACL 2020 (demo), 335–342.
- [18] I. M. Sobol’. *Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates*. Mathematics and Computers in Simulation, 55(1–3):271–280, 2001.